

UNIVERSITY "ALEXANDRU IOAN CUZA" OF IASI
FACULTY OF COMPUTER SCIENCE



DISSERTATION THESIS

The Longest Induced Path Problem

proposed by

Răzvan-Emanuel Apetrei

Session: July, 2026

Scientific Coordinator

Lect. Dr. Frăsinaru Cristian

UNIVERSITY "ALEXANDRU IOAN CUZA" OF IASI
FACULTY OF COMPUTER SCIENCE

The Longest Induced Path Problem

Răzvan-Emanuel Apetrei

Session: July, 2026

Scientific Coordonator

Lect. Dr. Frăsinaru Cristian

Avizat,
Îndrumător lucrare de licență,
Lect. Dr. Frăsinaru Cristian.

Data: Semnătura:

Declarație privind originalitatea conținutului lucrării de licență

Subsemnatul **Apetrei Răzvan-Emanuel** domiciliat în **România, județul Neamț, comuna Dumbrava Roșie, strada Zimbrului, numărul 159**, născut la data de **22 septembrie 2002**, identificat prin CNP **5020922270048**, absolvent al Faculty of Computer Science, Faculty of Computer Science specializarea **Computer Science**, promoția 2026, declar pe propria răspundere cunoscând consecințele falsului în declarații în sensul art. 326 din Noul Cod Penal și dispozițiile Legii Educației Naționale nr. 1/2011 art. 143 al. 4 și 5 referitoare la plagiat, că lucrarea de disertație cu titlul **The Longest Induced Path Problem** elaborată sub îndrumarea domnului **Lect. Dr. Frăsinaru Cristian**, pe care urmează să o susțin în fața comisiei este originală, îmi aparține și îmi asum conținutul său în întregime.

De asemenea, declar că sunt de acord ca lucrarea mea de disertație să fie verificată prin orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la introducerea conținutului ei într-o bază de date în acest scop.

Am luat la cunoștință despre faptul că este interzisă comercializarea de lucrări științifice în vederea facilitării falsificării de către cumpărător a calității de autor al unei lucrări de licență, de diplomă sau de disertație și în acest sens, declar pe proprie răspundere că lucrarea de față nu a fost copiată ci reprezintă rodul cercetării pe care am întreprins-o.

Data:

Semnătura:

Declarație de consimțământ

Prin prezenta declar că sunt de acord ca lucrarea de disertație cu titlul **The Longest Induced Path Problem**, codul sursă al programelor și celelalte conținuturi (grafice, multimedia, date de test, etc.) care însoțesc această lucrare să fie utilizate în cadrul Faculty of Computer Science.

De asemenea, sunt de acord ca Faculty of Computer Science de la University "Alexandru Ioan Cuza" of Iași, să utilizeze, modifice, reproducă și să distribuie în scopuri necomerciale programele-calculator, format executabil și sursă, realizate de mine în cadrul prezentei lucrări de disertație.

Absolvent **Răzvan-Emanuel Apetrei**

Data:

Semnătura:

Contents

Motivation	2
Introduction	3
1 Existing approaches	7
1.1 Integer programming formulations of Matsypura et al.	7
1.1.1 Exact algorithm	8
1.1.2 Randomized heuristic	10
1.2 ILP formulations of Bökler et al.	10
1.2.1 Extended graph and base formulation	10
1.2.2 Cut constraints and clique inequalities	12
1.3 Branch-and-cut formulations of Marzo et al.	13
1.3.1 Formulation with explicit cycle elimination constraints	13
1.3.2 Formulation with cutset constraints	14
1.3.3 Clique inequalities	14
1.3.4 Branch-and-cut implementation	15
1.4 Backtracking algorithms of Marzo and Ribeiro	15
1.4.1 Exact enumerative algorithm	16
1.4.2 HLIPP heuristic	17
1.4.3 Greedy variant G-HLIPP	20
2 Solution Description	21
2.1 Ant Colony Optimization Algorithm	22
2.1.1 Algorithm overview	23
2.1.2 Path construction via pheromone-guided DFS	25
2.1.3 Candidate selection	26
2.1.4 Pheromone model	27

2.1.5	Pheromone operations	28
2.1.6	Stagnation management	29
2.1.7	Parallelism	30
2.2	Genetic Algorithm	31
2.2.1	Algorithm overview	32
2.2.2	Individual representation	33
2.2.3	Population initialization	34
2.2.4	Core DFS with backtracking	35
2.2.5	Bidirectional DFS	36
2.2.6	Path extension	37
2.2.7	Mutation operator	38
2.2.8	Crossover operator	39
2.2.9	Selection operator	41
2.2.10	Fresh individual injection	42
2.2.11	Stagnation restart	43
3	Computational Experiments	44
3.1	Test Instances	44
3.1.1	Instance generator	45
3.2	Computational Environment	46
3.3	Results	46
3.3.1	Small and medium instances	47
3.3.2	Large real-world instances	47
3.3.3	Generated instance	49
3.3.4	Algorithm comparison	49
4	Concluding remarks	51
4.1	Ethical AI Usage	51
4.2	Conclusion	51
4.3	Future Work	53
	Bibliography	55

Motivation

The Longest Induced Path problem, abbreviated LIP, requires finding the maximum cardinality subset of vertices such that it induces a simple path. This means that no shortcut edges exist between non-consecutive vertices which makes it substantially harder than the classical longest path problem which is also NP-complete. There is no known constant-factor polynomial-time approximation for general graphs and it remains NP-complete even on bipartite graphs [5].

LIP has practical applications in several domains. In communication and transportation networks, it calculates the worst-case distance between two nodes when intermediate vertices can fail, become overloaded, or be taken out by an adversary which forces traffic through longer alternative routes [4]. This is useful in practice when roads for example can have a lot of traffic and alternate routes must be used. The problem is also connected to the snake-in-the-box problem, which has applications in the theory of error-correcting codes [6].

Yet solution methods for this problem are still limited. Existing approaches rely on integer linear programming formulations (ILP) [4, 7, 1] which do not scale to large graphs beyond several hundred vertices and a few heuristics based on an improved version of backtracking [1]. There are no attempts to use population-based metaheuristics which have proven effective for other related NP-hard graph optimization problems.

This is what motivated us to explore new approaches, this thesis presents three new approaches, an ILP formulation with lazy subtour elimination, a hybrid genetic algorithm, an Ant Colony Optimization variant based on directional edge pheromone. We benchmark on 24 real-world graphs and additional instances produced by a custom generator. The results show that these metaheuristic methods outperform existing approaches on large instances that can be used in practice.

Introduction

Graph theory is an important and fundamental framework used to represent and solve a large variety of complex problems in mathematics, computer science, biology and many other fields. Graphs are data structures which consist of a set of vertices (also called nodes) and edges used to connect these vertices. Each vertex represents a concept or an entity, while edges represent the relationships or interactions between them. Throughout this thesis, I will be using undirected graphs where each edge is bidirectional, implying that the relationship it represents is a symmetric one.

To present a graph G , we use two sets: V and E . $V = \{0, 1, \dots, n-1\}$ is the set of vertices, where n denotes the total number of vertices in the graph and $E \subseteq V \times V$ is the set containing pairs of vertices that are connected by an edge.

A key concept in graph theory is that of a path, in particular, an induced path. We will denote with $S \subseteq V$, a subset of vertices from V . If we keep only the vertices in S and the edges in E that connect the vertices within S , from a graph $G = (V, E)$, then we will obtain the induced subgraph $G[S] = (S, E')$. If this subgraph forms a simple path, then we have found an induced path within the original graph. In other words, if the vertices we have chosen have no shortcut edges, the only edges being those of consecutive vertices, then the subgraph of these vertices will be a path.

The Longest Induced Path problem is NP-complete and in practice is difficult to work with since the candidate solutions are fragile. In the standard longest path problem, once we have a valid path, extending it only requires finding an unvisited neighbor of one of the endpoints. With induced paths, extending the current induced path with a new vertex can invalidate the entire solution if the vertex chosen has an edge to an existing vertex in the graph except the current endpoint. As a result, the search space is full of near-solutions that are not actually feasible, which makes both exact and heuristic methods struggle on larger instances. This also makes it difficult for many heuristics to ensure that the solutions produced are induced paths.

To prove that LIP is NP-complete, we first check that it belongs to NP. Given a list of vertices claimed to form an induced path, we can verify in polynomial time that all vertices are distinct, that consecutive ones are connected by edges, and that no two non-consecutive vertices are adjacent. For NP-hardness, we reduce from the Longest Path problem, which is NP-complete since Hamiltonian Path is the special case where $k = |V|$. Given an instance (G, k) of Longest Path, we build a new graph G' by subdividing every edge once: for each edge $uv \in E(G)$, we remove uv , add a new vertex x_{uv} , and add the two edges ux_{uv} and $x_{uv}v$. We then ask whether G' contains an induced path on at least $K = 2k - 1$ vertices. If G has a simple path $v_1, v_2, \dots, v_k$, then in G' we get the path $v_1, x_{12}, v_2, x_{23}, \dots, x_{k-1,k}, v_k$ on $2k - 1$ vertices, and this path is induced because after subdivision there are no direct edges between original vertices and each subdivision vertex connects only to its two original endpoints. Conversely, if G' has an induced path on at least $2k - 1$ vertices, then since G' is bipartite (every edge connects an original vertex to a subdivision vertex), the path must alternate between the two sides, and reading off the original vertices in order gives a simple path of at least k vertices in G . Since G' is bipartite, this also shows that LIP remains NP-complete even for bipartite graphs [5].

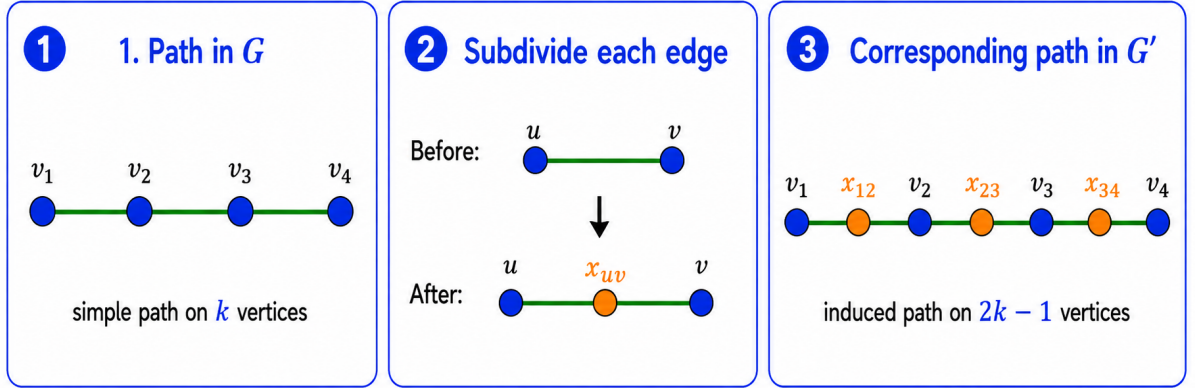


Figure 1: Example converting from Longest Path to LIP (generated using GPT-5.5)

My primary motivation for this work comes from the lack of metaheuristic approaches in the literature for this problem. Genetic algorithms and swarm intelligence methods have been applied to many NP-complete graph problems such as graph coloring, maximum clique, and the traveling salesman problem, but I have not found any

attempts to use them for LIP. The existing solution methods are limited to integer linear programming formulations [4, ?, 1], which do not scale beyond several hundred vertices, and a heuristic based on an improved version of backtracking [1]. This leaves a gap, especially for large real-world graphs where exact methods become intractable. LIP also has practical applications in several domains. In communication and transportation networks, it gives the worst-case distance between two nodes when intermediate vertices can fail, become overloaded, or be taken out by an adversary, forcing traffic through longer alternative routes [4]. In social networks with information cascades, the longest induced path is the longest possible chain along which a piece of information can spread from one person to the next, with the originator at one end and the last recipient at the other. The problem is also connected to the snake-in-the-box problem, which has applications in the theory of error-correcting codes [6].

Solving LIP on large graphs is difficult in practice. The induced path constraint makes the search space much more restrictive than for the standard longest path problem: adding a single vertex to a candidate path can invalidate it entirely if that vertex has edges to non-consecutive vertices already on the path. Exact methods like ILP can provide provably optimal solutions, but their running time grows quickly with graph size. In our experiments, ILP solved all instances with up to 662 vertices within five minutes, but on the largest instance with 2,361 vertices, it could only find a path of 4 vertices in two minutes. This is what makes metaheuristic approaches necessary for larger graphs. Designing these metaheuristics for LIP also proved difficult. Genetic algorithms require a representation and repair mechanism that can turn arbitrary solutions into valid induced paths. Ant Colony Optimization needs a pheromone model that captures the structure of good induced paths, and as I discovered during this work, the direction in which an edge is traversed matters because the same edge can be beneficial in one direction but harmful in the other due to different induced path constraints on each side.

This thesis extends my bachelor’s work, where I developed an initial genetic algorithm for LIP, with a completely redesigned set of algorithms. The main contributions are: an ILP formulation with lazy subtour elimination that finds optimal solutions for all tested instances up to 662 vertices, where formulations from the literature timed out on the same instances; a tuned genetic algorithm with a hybrid variant that uses ILP solutions to seed the initial population; and an Ant Colony Optimization algorithm that introduces directional edge pheromone for LIP, which is to the best of my knowledge

the first ACO approach for this problem. I also developed a custom instance generator that creates graphs with planted induced paths of known length, which allows testing algorithms under controlled conditions. We benchmark these approaches on 24 real-world graph instances from social networks, power grids, ecological networks, and protein interaction networks, with sizes ranging from 33 to 2,361 vertices.

The rest of this thesis is organized as follows. Chapter 1 reviews the existing approaches for solving LIP, including the integer programming formulations of Matsypura et al. [4] and Bökler et al. [?], and the exact enumerative algorithm and heuristic of Marzo and Ribeiro [1]. Chapter 2 details the three proposed algorithms: the ILP formulation, the genetic algorithm with its hybrid variant, and the ACO algorithm with directional edge pheromone, along with their design decisions and parameter choices. Chapter 3 documents the computational experiments and analyzes the performance of each algorithm across all instances. It also includes the custom instance generator developed for controlled testing and the benchmark instances used. Chapter 4 discusses the role of artificial intelligence tools in the development of this thesis and also presents the conclusions and directions for future work.

Chapter 1

Existing approaches

In this chapter, I will present the existing state of the art for this problem. While there are no scientific papers to solve this using any meta-heuristics, there are still a few papers to solve it exact algorithms and a few random heuristics. The exact algorithms, while useful as they guarantee optimal solutions, are not practical for large instances. The heuristics on the other side don't guarantee finding the exact solution but they can handle larger graphs. The approaches are presented in chronological order, starting with the integer programming formulations of Matsypura et al. [4], continued by stronger ILP formulations of Bökler et al. [7], the branch-and-cut procedures of Marzo, Melo, Ribeiro, and Santos [2], and finally the backtracking algorithms of Marzo and Ribeiro [1].

1.1 Integer programming formulations of Matsypura et al.

Matsypura et al. [4] were the first to develop exact solution approaches for the longest induced path problem on general graphs. They proposed four integer programming (IP) formulations based on three conceptually different interpretations of the problem, along with a randomized heuristic for larger instances. Two of the four formulations, referred to as IP3 and IP3c, are summarized below, as they form the basis on which later formulations were built.

1.1.1 Exact algorithm

The key idea of the linear programming formulations proposed by Matsypura et al. [4] were to create a walk in the graph and then force it to be an induced path using additional constraints. A walk is a sequence of vertices in a graph, visited one after another where each pair of consecutive vertices has to be connected by an edge.

We start the walk at some vertex at time $t = 0$ and then move to one of its neighbors at time $t = 1$. We continue adding vertices until $t = T$ or no further valid moves are available. The parameter T which is the upper bound on the length of any induced path can be set to $|V| - 1$ unless we know a better lower bound.

To model this walk as an ILP formulation, we use a binary variable x_i^t for each vertex $i \in V$ which for each time step $t \in \{0, \dots, T\}$:

$$x_i^t = \begin{cases} 1, & \text{if and only if vertex } i \text{ is visited at time } t, \\ 0, & \text{otherwise.} \end{cases}$$

At each time step, the walk visits a new vertex whenever possible. If the walk cannot continue at some time $t = r$, then $x_i^t = 0$ for all $i \in V$ and $t \geq r$. To guarantee that the walk is valid in this graph, we have to ensure that we have used only the edges in E and to ensure it is an induced path, we have to check that there are no shortcuts to previously visited vertices. The IP3 formulation is:

$$(IP3): \quad \max \sum_{t=0}^T \sum_{i \in V} x_i^t \tag{2.1}$$

subject to:

$$\sum_{i \in V} x_i^t \leq 1, \quad \forall t \in \{0, \dots, T\}, \tag{2.2}$$

$$\sum_{t=0}^T x_i^t \leq 1, \quad \forall i \in V, \tag{2.3}$$

$$x_i^t + x_j^{t+1} \leq 1, \quad \forall (i, j) \notin E, \forall t \in \{0, \dots, T-1\}, \tag{2.4}$$

$$x_i^t + x_j^\tau \leq 1, \quad \forall (i, j) \in E, \forall t \in \{0, \dots, T-2\}, \forall \tau \in \{t+2, \dots, T\}, \tag{2.5}$$

$$\sum_{i \in V} x_i^t \leq \sum_{i \in V} x_i^{t-1}, \quad \forall t \in \{1, \dots, T\}, \tag{2.6}$$

$$x_i^t \in \{0, 1\}, \quad \forall i \in V, \forall t \in \{0, \dots, T\}. \tag{2.7}$$

The objective function (2.1) maximizes the number of vertices in the walk. Constraints (2.2) ensure we visit only one vertex at any given time. Constraints (2.3) pre-

vent the walk from containing the same vertex more than once. Constraints (2.4) require that the walk traverse only existing edges in E . Constraints (2.5) ensures the induced property of the path by preventing any shortcuts: If vertex i is visited at time t and vertex j is a neighbor of i , then we can pick j only at time $t + 1$ (which is directly after i), not at any later time $\tau \geq t + 2$ which would create a shortcut. Constraints (2.6) prevent the walk from restarting after being interrupted: if no vertex is visited at time $t - 1$, then no vertex can be visited at time t either. Finally, constraints (2.7) impose binary integrality on all variables.

This formulation has $O(T|V|)$ variables and $O(T|V|^2 + T^2|E|)$ constraints. This formulation can be improved by aggregating constraints (2.4) and (2.5) which would reduce the constraint count. This new more compact formulation is called IP3c and has the same LP relaxation bound. Specifically, constraints (2.4) can be consolidated as:

$$\sum_{j \in V \setminus \{(i,j) \in E\}} x_j^{t+1} \leq 1 - x_i^t, \quad \forall i \in V, \forall t \in \{0, \dots, T-1\}, \quad (2.8)$$

which states that if vertex i is visited at time t (i.e., $x_i^t = 1$), then none of its non-neighbors can be visited at time $t + 1$. When $x_i^t = 0$, the constraint holds because constraint (2.2) prevents x_i^t from having a value higher than one. Constraints (2.5) can be aggregated similarly:

$$\sum_{\tau=t+2}^T x_j^\tau \leq 1 - x_i^t, \quad \forall (i,j) \in E, \quad \forall t \in \{0, \dots, T-2\}, \quad (2.9)$$

since if vertex i is visited at time t , its neighbors can only be visited at time $t + 1$, not at any later time. Visiting them later would create a cycle, violating the induced path property. When $x_i^t = 0$, the constraint is satisfied by constraints (2.3), which prevent any vertex from being visited more than once.

The exact iterative algorithm proposed by Matsypura et al. solves a sequence of progressively larger IPs rather than trying to solve the full problem at once. The time parameter T must be set to $|V| - 1$ to guarantee validity of the IP3 and IP3c formulations, unless a better upper bound is known. However since T is an upper bound, the exact algorithm can find an induced path L^* that is less than T . In this scenario, L^* is the optimal value and the path found is the longest. Since every graph has an induced path whole length equals its diameter, the iterative search can start from $T = \text{diam}(G) + 1$. At each step, if $L^* = T$ then we increment T and run the integer formulation again only stopping when we get an induced path of length less than T or when T reached

$|V| - 1$. When L^* is much smaller than $|V|$, each IP in this sequence is considerably faster to solve than trying to solve the full formulation with $T = |V| - 1$.

1.1.2 Randomized heuristic

Since exact methods become impractical for larger instances, Matsypura et al. [4] proposed a randomized heuristic based on random walks. The algorithm generates random walks which are also induced starting from each vertex $s \in S \subseteq V$, where S is a predefined subset of vertices. We initialize the incumbent as an empty path, and then, for each vertex $s \in S$, the heuristic generates k random walks from s , with k given as a parameter. The longest path found among these k random walks is returned, the incumbent is updated if it is smaller, and a new vertex $s \in S$ is explored. The incumbent will contain the longest induced path found after all vertices in S are processed by the heuristic. There are two issues with this algorithm, first k and S are not dynamically adjusted based on the graph characteristics, leading to many repeated paths starting from the same vertex for smaller graphs or insufficient exploration in larger ones. The second issue is that there is no logic guiding the generation of these walks. Metaheuristics manage to find subsets of vertices that have potential to find larger induced paths and improve upon them while still exploring other vertices. Algorithms that are just random have no direction in the search space of the solutions leading to a very low probability of finding the best solution if the search space is massive.

1.2 ILP formulations of Bökler et al.

Bökler et al. [7] proposed stronger integer linear programming formulations for the longest induced path problem, using constraints based on induced cycles by adding a new vertex s to the initial graph. These formulations use cut (or generalized subtour elimination) constraints and clique inequalities. Their formulations produce tighter LP relaxation bounds than those of Matsypura et al. [4].

1.2.1 Extended graph and base formulation

The approach of Bökler et al. is built on an extended graph $G_s = (V_s, E_s)$, constructed by adding to G a new universal vertex s that is adjacent to every vertex in

V :

$$V_s = V \cup \{s\}, \quad E_s = E \cup \{(s, v) : v \in V\}.$$

With the help of this new vertex, we can create a formulation that searches for a longest induced cycle through s in G_s instead of an induced path. Any induced cycle through s corresponds to an induced path in the original graph G , the two neighbors of s in the cycle being the two endpoints of the path. This simplifies the constraints, since every vertex on a cycle has a degree of exactly two in the cycle.

Instead of having a variable for every vertex in the graph, this formulation uses a binary edge variable y_e for each edge $e \in E_s$:

$$y_e = y_{i,j} = \begin{cases} 1, & \text{if edge } e = (i, j) \text{ is selected,} \\ 0, & \text{otherwise.} \end{cases}$$

The base formulation, called ILP Base, is:

$$\text{(ILP Base):} \quad \max \sum_{e \in E_s} y_e \tag{2.10}$$

$$\sum_{v \in V} y_{s,v} = 2, \tag{2.11}$$

$$2y_e \leq \sum_{f \in \delta(e)} y_f \leq 2, \quad \forall e \in E, \tag{2.12}$$

$$y_e \in \{0, 1\}, \quad \forall e \in E_s. \tag{2.13}$$

The first constraint, called the objective function (2.10), maximizes the number of selected edges, which is equal to the number of vertices in the induced cycle. Constraint (2.11) forces the induced cycle to use the universal vertex s by requiring exactly two edges incident to this vertex (the degree is 2 since we are searching for a cycle). Constraints (2.12) enforce two conditions simultaneously through a two-sided inequality. The right inequality ($\sum_{f \in \delta(e)} y_f \leq 2$) states that for any edge $e \in E$ from the original graph, at most two of its adjacent edges can be selected. This is the constraint that ensures that the cycle produced is an induced cycle. In order to explain this further, we will consider the two scenarios, one where e is selected and one where e is not selected. When e is selected, this limits its endpoints to a degree of two which maintains the cycle structure. On the other hand, when e is not selected, if both endpoints of a non selected edge are interior vertices of the path, they would each contribute at least two selected adjacent edges which would exceed the limit of two. This prevents any shortcuts in the path and any chords in the cycle. The left inequality ($2y_e \leq \sum_{f \in \delta(e)} y_f$)

ensures that if an edge e is selected, then at least two of its adjacent edges are also selected, giving each internal vertex degree two on the cycle. The last constraint (2.13) is required for binary integrality.

1.2.2 Cut constraints and clique inequalities

The ILP Base formulation alone is insufficient since it allows solutions consisting of multiple disjoint cycles, only one of which passes through s . We will have to add another constraint to force the solution into a single cycle through s . Bökler et al. [7] added a new cut constraint to ILP Base. Let $\delta(W) = \{(w, w') \in E_s : w \in W, w' \in V_s \setminus W\}$ denote the set of edges in the cut induced by $W \subseteq V$. The cut constraints are:

$$\sum_{e \in \delta(v)} y_e \leq \sum_{e \in \delta(W)} y_e, \quad \forall W \subseteq V, \quad \forall v \in W. \quad (2.14)$$

These constraints guarantee the connectivity of the solution. For each vertex v incident to a selected edge, if we create a cut that separating it from s then this constraint forces us to have at least two paths between v and s in the subgraph (since the left side of the equation is equal to two for a selected edge). If we combine this with the other constraints of ILP Base which guarantee the induced cycle property, this will ensure that we will get an induced cycle going through s and by removing s from the end solution, we will obtain the longest induced path in the graph. This formulation is called ILP Cut.

The formulation can be further improved with clique inequalities. These are used to reduce the searching space of the problem. If a subset $Q \subseteq V$ induces a clique in G , then any induced path can contain at most one edge of the clique:

$$\sum_{e \in E[Q]} y_e \leq 1, \quad \forall Q \subseteq V \text{ such that } G[Q] \text{ is a clique.} \quad (2.15)$$

In order to find the best performing formulation, Bökler et al. [7] tested a multitude of configurations. They've tried the use of clique constraints, fractional separation and the introduction of vertex variables. Through various tests and benchmarks, they concluded that the best uses vertex variables (with edge variables relaxed to continuous) with clique inequalities for all maximal cliques and separates the cut constraints only for integer solutions. They've named this $C_{\text{int}}^{m,c}$, the other alternative variant C_{int}^m omits the clique inequalities. Computational experiments showed that these formulations are significantly stronger than the time-indexed formulations previously proposed by Matsypura et al. on most instance classes.

1.3 Branch-and-cut formulations of Marzo et al.

In a follow-up paper, Marzo, Melo, Ribeiro, and Santos [2] proposed two new integer programming formulations for the longest induced path problem which are essentially improvements to the original proposed algorithms of Bökler et al. [7]. These have an exponential number of constraints so they are solved using branch-and-cut procedures. Once again, we create the extended graph $G_s = (V_s, E_s)$ with a universal vertex s adjacent to all vertices of V , and search for the longest induced cycle through s .

Compared to the previous formulations, we will have binary variables for both vertices and edges at the same time. So, the formulations use binary vertex variables y_v for each $v \in V_s$ and binary edge variables x_e for each $e \in E_s$:

$$y_v = \begin{cases} 1, & \text{if vertex } v \text{ belongs to the solution,} \\ 0, & \text{otherwise,} \end{cases} \quad x_e = \begin{cases} 1, & \text{if edge } e \text{ is in the solution,} \\ 0, & \text{otherwise.} \end{cases}$$

1.3.1 Formulation with explicit cycle elimination constraints

The first formulation, called *cec*, enforces acyclicity by explicitly forbidding all cycles. Let $\mathcal{C}$ denote the family of all cycles in G . The formulation is:

$$\text{(cec): } \max \sum_{v \in V} y_v \tag{2.16}$$

$$\text{s.t. } \sum_{e \in \delta_{G_s}(v)} x_e = 2y_v, \quad \forall v \in V, \tag{2.17}$$

$$\sum_{e \in \delta_{G_s}(s)} x_e = 2, \tag{2.18}$$

$$\sum_{v \in C} y_v \leq |C| - 1, \quad \forall C \in \mathcal{C}, \tag{2.19}$$

$$x_e \leq y_v, \quad \forall v \in V, e \in \delta_{G_s}(v), \tag{2.20}$$

$$x_e \geq y_u + y_v - 1, \quad \forall e = uv \in E, \tag{2.21}$$

$$x \in \{0, 1\}^{|E_s|}, y \in \{0, 1\}^{|V_s|}. \tag{2.22}$$

The objective (2.16) maximizes the number of vertices in the induced path. Constraints (2.17) enforce that every selected vertex has degree exactly two in the solution, meaning it has exactly two incident selected edges, making it an interior vertex of the cycle through s . Constraint (2.18) requires exactly two edges incident to s , corresponding to the two endpoints of the induced path. Constraints (2.19) are the cycle

elimination constraints: for every cycle C in G , at most $|C| - 1$ of its vertices can be selected, which prevents the solution from containing a cycle that does not pass through s . Constraints (2.20) link the edge and vertex variables by ensuring that a selected edge can only be incident to a selected vertex. Constraints (2.21) enforce the induced path property: if both endpoints of an edge $e = uv \in E$ are selected (i.e., $y_u = y_v = 1$), then e must also be selected. This means that no chord can exist between two non-consecutive vertices on the path. Together, constraints (2.20) and (2.21) guarantee that the selected edges form exactly the subgraph induced by the selected vertices.

1.3.2 Formulation with cutset constraints

The second formulation, called *cut*, replaces the cycle elimination constraints with cutset constraints that enforce connectivity. It uses the same objective function and constraints (2.16)–(2.18) and (2.20)–(2.22), but replaces (2.19) with:

$$\sum_{e \in \delta_{G_s}(S)} x_e \geq 2y_v, \quad \forall S \subseteq V, \forall v \in S, \quad (2.23)$$

where $\delta_{G_s}(S)$ denotes the set of edges with one endpoint in S and the other in $V_s \setminus S$. These constraints guarantee that if any vertex v in a subset S is selected, then the cut separating S from s must contain at least two selected edges. This forces the solution to be connected through s . Since a connected solution satisfying the degree constraints (2.17) and (2.18) must form a single cycle through s , connectivity alone is sufficient to prevent spurious cycles.

1.3.3 Clique inequalities

Marzo et al. considered two forms of clique inequalities. Bökler et al. originally used edge-based clique inequalities (2.15): for every clique $K \subseteq V$, the number of selected edges within K is at most one. Marzo et al. additionally considered vertex-based clique inequalities:

$$\sum_{v \in K} y_v \leq 2, \quad \forall K \subseteq V \text{ such that } G[K] \text{ is a clique.} \quad (2.24)$$

These state that at most two vertices from any clique can appear on an induced path, since a path can enter and leave a clique at most once. Marzo et al. proved that within their formulations, neither form of clique inequality implies the other in general, and they characterized the conditions under which one implies the other.

1.3.4 Branch-and-cut implementation

Since both formulations have an exponential number of constraints (one for every cycle in $\mathcal{C}$, or one for every subset $S \subseteq V$ and vertex $v \in S$), Marzo et al. solve them using branch-and-cut, where violated constraints are separated on the fly during the branch-and-bound process. The separation procedures are:

- **Cycle constraints** (for *cec*): the separation graph G_{sep} is the subgraph induced by the vertices with $\hat{y}_v > 0$ in the current solution. For integer solutions, a depth-first search on G_{sep} detects all cycles via back edges. For fractional solutions, a heuristic variant of DFS visits vertices in decreasing order of their $\hat{y}$ values, adding any violated cycle constraints found during the search.
- **Cutset constraints** (for *cut*): for integer solutions, a breadth-first search from s identifies disconnected components; each vertex in a component not reachable from s yields a violated cutset constraint. For fractional solutions, maximum flow computations from s to each vertex in G_{sep} are used to find minimum cuts, following the exact separation approach of Magnanti and Wolsey [?].
- **Clique inequalities**: when the number of maximal cliques with at least three vertices does not exceed 500, all corresponding clique inequalities are added to the formulation upfront (the maximal cliques are enumerated using the Bron-Kerbosch algorithm). Otherwise, a greedy heuristic separation procedure constructs maximal cliques by iteratively adding vertices with the highest $\hat{y}$ values, breaking ties by degree, and lifts violated inequalities by extending the clique in the original graph.

All separation procedures for fractional solutions are executed only at the root node of the branch-and-bound tree, so as not to overload the formulation with inequalities generated throughout the search tree.

1.4 Backtracking algorithms of Marzo and Ribeiro

Marzo and Ribeiro [1] proposed both an exact algorithm and a heuristic based on enumerating induced paths using backtracking. Their approach, while conceptually simpler than the ILP methods, proved competitive with the integer programming formulations on many instance classes.

1.4.1 Exact enumerative algorithm

The exact algorithm uses recursion to generate all induced paths starting from each vertex in the graph. It consists of two procedures: **EXACT-ENUM**, which iterates over starting vertices, and **INDUCED-PATHS-FROM-VERTEX**, which recursively builds induced paths from a given source.

The first procedure, **EXACT-ENUM** (Algorithm 1), takes the graph as input. Lines 1 and 2 initialize two variables: $P_{\max}$, storing the longest induced path found so far, and P_{temp} , holding the current path under construction. The loop in lines 3–5 iterates through all vertices of G , calling **INDUCED-PATHS-FROM-VERTEX** for each vertex $s \in V$ as a potential starting point.

Algorithm 1 EXACT-ENUM(G)

```
1:  $P_{\max} \leftarrow \emptyset$ 
2:  $P_{\text{temp}} \leftarrow \emptyset$ 
3: for each  $s \in V$  do
4:   INDUCED-PATHS-FROM-VERTEX( $G, s, P_{\max}, P_{\text{temp}}$ )
5: end for
6: return  $P_{\max}$ 
```

The second procedure called **INDUCED-PATHS-FROM-VERTEX** (Algorithm 2) takes as inputs the graph G , the source vertex s , the longest induced path found so far $P_{\max}$ and the current path P_{temp} . The first line adds the source vertex s to the current path P_{temp} we are creating. Line 2 constructs the set $N(s)$ containing the vertices adjacent to s in the graph G . If this set is not empty then lines 4-9 iterates through this set. For each vertex t , lines 5-7 create the temporary subgraph G_{temp} by removing s and its neighbors, except t . This has the purpose of preventing visiting in the future any node that could potentially create a cycle, ensuring that the path under construction is an induced path and reducing the searching space significantly. The loop finishes with line 8 which calls recursively the current procedure for the new induced graph G_{temp} with t as the new source vertex, repeating the process again. In the scenario where vertex s has no neighbours ($N(s)$ is empty), it means we cannot extend any further the current path. In this case, lines 11-13 are used to evaluate the current path. If the size of the current path P_{temp} is greater than the longest induced path found so far $P_{\max}$ then we update the value of $P_{\max}$ to P_{temp} . Before backtracking and moving to the next vertex, the procedure concludes with line 15 which removes the source vertex s from the current path ensuring that subsequent paths are correctly constructed without the

previously added vertices.

When $N(s)$ is empty, the current path cannot be extended further. Lines 11–13 compare P_{temp} with P_{max} and update P_{max} if the current path is longer. Line 15 removes s from P_{temp} as the recursion unwinds, restoring the path for subsequent branches.

Algorithm 2 INDUCED-PATHS-FROM-VERTEX($G, s, P_{\text{max}}, P_{\text{temp}}$)

```

1:  $P_{\text{temp}} \leftarrow P_{\text{temp}} \oplus \{s\}$ 
2:  $N(s) \leftarrow \{t : (s, t) \in E, t \in V\}$ 
3: if  $N(s) \neq \emptyset$  then
4:   for each  $t \in N(s)$  do
5:      $V_{\text{temp}} \leftarrow V \setminus (\{s\} \cup N(s) \setminus \{t\})$ 
6:      $E_{\text{temp}} \leftarrow \{(i, j) \in E : i \in V_{\text{temp}}, j \in V_{\text{temp}}\}$ 
7:      $G_{\text{temp}} \leftarrow (V_{\text{temp}}, E_{\text{temp}})$ 
8:     INDUCED-PATHS-FROM-VERTEX( $G_{\text{temp}}, t, P_{\text{max}}, P_{\text{temp}}$ )
9:   end for
10: else
11:   if  $|P_{\text{temp}}| > |P_{\text{max}}|$  then
12:      $P_{\text{max}} \leftarrow P_{\text{temp}}$ 
13:   end if
14: end if
15:  $P_{\text{temp}} \leftarrow P_{\text{temp}} \ominus \{s\}$ 

```

Computational experiments in [1] showed that EXACT-ENUM is faster than the exact algorithm of Matsypura et al. on the RWC instances and competitive with the best ILP Cut implementations of Bökler et al. [7]. On the 773 MG instances, the enumerative algorithm solved all of them with the smallest running times among all tested methods. However, it does not perform well on the BAL instances with 100 vertices and higher edge densities, where the ILP-based methods have an advantage.

1.4.2 HLIPP heuristic

The exact algorithm works well for smaller instances but becomes impractical for larger ones, as expected for any exact method applied to an NP-complete problem. Marzo and Ribeiro addressed this by proposing a heuristic called **HLIPP** (Heuristic Longest Induced Path Problem), which limits the number of paths generated from each starting vertex.

The first procedure, **HLIPP** (Algorithm 3), follows the same structure as EXACT-ENUM. Lines 1 and 2 initialize P_{max} and P_{temp} . The loop in lines 3–8 iterates through all vertices. For each vertex s , lines 4–6 initialize three variables: $\#paths$, counting the number of maximal induced paths generated from s ; $last_improv$, recording the path

count at the time of the last improvement; and *truncated*, a flag indicating whether the search from s has been stopped. Line 7 calls the recursive search procedure **FIRST-INDUCED-PATHS**.

Algorithm 3 HLIPP($G, \text{maxpaths}$)

```

1:  $P_{\max} \leftarrow \emptyset$ 
2:  $P_{\text{temp}} \leftarrow \emptyset$ 
3: for each  $s \in V$  do
4:    $\#paths \leftarrow 0$ 
5:    $last\_improv \leftarrow 0$ 
6:    $truncated \leftarrow \text{False}$ 
7:   FIRST-INDUCED-PATHS( $G, s, P_{\max}, P_{\text{temp}}, \text{maxpaths}, \#paths, last\_improv,$ 
                         $truncated$ )
8: end for
9: return  $P_{\max}$ 

```

The second procedure, **FIRST-INDUCED-PATHS** (Algorithm 4), carries out the bounded search. It takes as input the graph G , the source vertex s , the best path $P_{\max}$, the current path P_{temp} , the stopping parameter maxpaths , the path counter $\#paths$, the last improvement counter $last_improv$, and the truncation flag $truncated$. Line 1 appends s to P_{temp} , and line 2 computes $N(s)$. If $N(s)$ is nonempty, the loop in lines 4–12 visits each neighbor t of s , constructing the reduced subgraph and recursing as in the exact algorithm (lines 5–8). If the recursive call sets $truncated$ to True, lines 9–11 propagate the truncation upward, stopping the search from the current vertex.

When $N(s)$ is empty, a maximal path has been reached. Line 14 increments $\#paths$. Lines 15–17 update $P_{\max}$ and $last_improv$ if the current path is an improvement. Lines 19–23 implement the stopping criterion: if more than maxpaths paths have been generated since the last improvement, the search from the current starting vertex is terminated by setting $truncated$ to True. Finally, line 25 removes s from P_{temp} during backtracking.

The difference between HLIPP and the randomized heuristic of Matsypura et al. lies in the exploration strategy. The randomized heuristic generates a fixed number of random paths from each starting vertex, which amounts to diversification through randomization. HLIPP, by contrast, deterministically enumerates paths and stops only after a sequence of maxpaths consecutive paths fails to improve the incumbent. This approach tends to explore more paths and generate them more efficiently. Experiments in [1] showed that HLIPP is consistently faster and finds better solutions than the randomized heuristic across all test instances. With $\text{maxpaths} = 10,000$, HLIPP found the

Algorithm 4 FIRST-INDUCED-PATHS($G, s, P_{\max}, P_{\text{temp}}, \text{maxpaths}, \#paths, \text{last_improv}, \text{truncated}$)

```

1:  $P_{\text{temp}} \leftarrow P_{\text{temp}} \oplus \{s\}$ 
2:  $N(s) \leftarrow \{t : (s, t) \in E, t \in V\}$ 
3: if  $N(s) \neq \emptyset$  then
4:   for each  $t \in N(s)$  do
5:      $V_{\text{temp}} \leftarrow V \setminus (\{s\} \cup N(s) \setminus \{t\})$ 
6:      $E_{\text{temp}} \leftarrow \{(i, j) \in E : i \in V_{\text{temp}}, j \in V_{\text{temp}}\}$ 
7:      $G_{\text{temp}} \leftarrow (V_{\text{temp}}, E_{\text{temp}})$ 
8:     FIRST-INDUCED-PATHS( $G_{\text{temp}}, t, P_{\max}, P_{\text{temp}}, \text{maxpaths}, \#paths, \text{last\_improv}, \text{truncated}$ )
9:   if  $\text{truncated} = \text{True}$  then
10:    return
11:  end if
12: end for
13: else
14:    $\#paths \leftarrow \#paths + 1$ 
15:   if  $|P_{\text{temp}}| > |P_{\max}|$  then
16:      $P_{\max} \leftarrow P_{\text{temp}}$ 
17:      $\text{last\_improv} \leftarrow \#paths$ 
18:   end if
19:   if  $\#paths - \text{last\_improv} > \text{maxpaths}$  then
20:      $P_{\text{temp}} \leftarrow \emptyset$ 
21:      $\text{truncated} \leftarrow \text{True}$ 
22:   return
23: end if
24: end if
25:  $P_{\text{temp}} \leftarrow P_{\text{temp}} \ominus \{s\}$ 

```

optimal solution for 32 out of 37 instances with known optima, in running times no greater than those of the exact algorithm.

1.4.3 Greedy variant G-HLIPP

Marzo and Ribeiro [1] also investigated which vertex properties correlate with the endpoints of optimal solutions. They found that 66.5% of the distinct endpoints of optimal paths have eccentricity equal to the graph diameter, while 98.2% have eccentricity at least $\text{diam}(G) - 1$. Similarly, 78.3% of endpoints have degree between 1 and 5, and 89.9% have degree below the average.

These observations led to an improved variant called **G-HLIPP**, which processes vertices in non-increasing order of their eccentricities, breaking ties in favor of vertices with smaller degree. The rationale is that vertices with high eccentricity and low degree are more likely to be endpoints of optimal paths, so exploring them first should find good solutions earlier. Computational experiments showed that G-HLIPP reached the same target solution values as HLIPP in shorter time for about 65% of the test runs.

Chapter 2

Solution Description

In this chapter, I present the two algorithms I propose to solve the longest induced path problem: an ant colony optimization (ACO) algorithm and a genetic algorithm. These two algorithms work for any family of graphs and are meant to produce good results even for larger graphs in a short amount of time compared to exact algorithms or even integer linear formulations. The current literature around this problem is rather lackluster as there are no proposal for more complex metaheuristics such as genetic algorithms, ant colony optimizations, tabu search, and so on. Current metaheuristics either rely on random search, exact algorithm with limits or ILP formulations with branch-and-bound.

A challenge shared by any metaheuristic trying to solve this problem is the maintaining of an induced path throughout the search. Having an individual that is an induced path and trying to improve it by adding a new vertex can lead to this property no longer being respected. At the same time, the best solutions require choosing the right vertices from the start as choosing the wrong vertices can lead to the search space diminishing and reaching a local maximum very quickly. This requires any metaheuristic to be specifically modified to this problem, for example, the standard mutation, crossover and solution construction operators can produce invalid paths (paths with chords). To preserve the induced path property, any operator must only extend the path with a vertex that has exactly one neighbor in the current path, and that neighbor must be the endpoint being extended.

2.1 Ant Colony Optimization Algorithm

The Ant Colony Optimization (ACO) algorithm is a population-based metaheuristic inspired by the behavior of ants. In ant colonies, every ant leaves deposits pheromone on the trails they follow in order to guide themselves and other ants. When choosing the path to follow, the ants also preferentially choose the paths with higher pheromones. This creates a positive feedback loop where over time ants will create a path with the highest pheromone which will be eventually followed by all ants (for example finding the source of food or path home). In the algorithmic version, the pheromone is stored as a matrix that encodes information from the previous iterations and it is used to guide the search.

Adapting ACO to the longest induced path problem is not an easy task. In the classical applications of ACO such as traveling salesman problem (TSP), the ants perform only a forward walk, visiting one vertex after another until the tour is complete. In our problem, this would lead to dead ends very quickly as the constraints are highly restrictive. At any point during the construction, when we visit a vertex, we have to eliminate all the other neighbors of the current path. To counter this, a modified version of the forward walk was implemented, one that uses a depth first search (DFS) with backtracking with every decision being guided by pheromones. This lets ants backtrack when they find dead ends and prevents dead ends from forming prematurely and then storing pheromone on that path leading the other ants to follow the same dead ends.

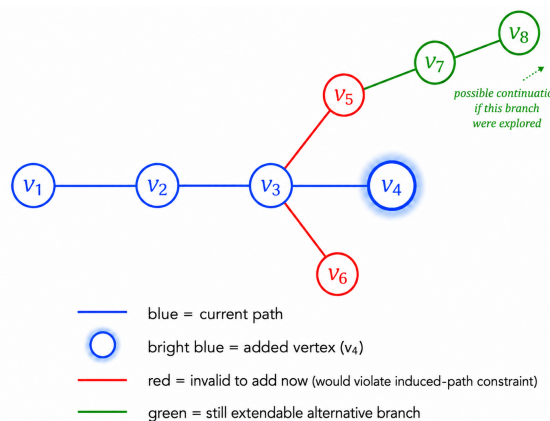


Figure 2.1: Example of extending an induced path

In the figure above, with blue I've denoted the current induced path. Upon

adding the vertex v_4 to this path, v_5 and v_6 are no longer available options as the construction moved to v_4 . Returning to v_3 while keeping v_4 in the path would result in an invalid path. It can be observed that in this scenario, choosing v_4 has lead us to a dead end as the other option of choosing v_5 had other nodes we could've chosen afterwards to further expand the path.

2.1.1 Algorithm overview

Algorithm 5 contains the overall structure of the ACO algorithm. This algorithm operates on each connected component for disconnected graphs and returns the best result.

Line 2 initializes the tasks array which contains the vertices which will be used as starting points for our ants. We choose only the vertices with a degree higher than 2 as the vertices with a lower degree requires no decisions to be taken. A vertex with degree of two is used just to extend the path further being in the middle of the path and one with degree of one can only be used as an endpoint of the path. These will be added to the path during the DFS search. Lines 3-5 initialize the variables: the pheromone matrix with a minimum pheromone on each edge, the best path and other variables with 0 or empty value. Lines 6-34 represent the main loop which stops when the time limit has been reached or when we have stagnated for many iterations. This is used to prevent the algorithm from running for the full time limit for easy to solve instances.

Lines 7-11 runs the pheromone guided DFS for each starting vertex in the tasks array. We add these paths to the *iterPaths* variable to be later used. Lines 13-27 handle the stagnation logic of the algorithm. If we have found an improvement, a path longer than the one we've found so far then we save it and reset the stagnation variables. On the contrary, if we haven't found a better path, we will increment stagnation and if we've reached maximum stagnation then we will restart the algorithm by resetting all pheromone values to default and adding the best path we've found to the tabu list. This list contains all paths that will be unable to deposit pheromone in the future so we don't get stuck in this local maximum again. If we stagnated enough times, line 25 increases the evaporation rate which will make the algorithm exploit less and explore more potentially finding better solutions in future iterations.

Last lines, 28-33, evaporate pheromone and deposit pheromone on the paths found. We first evaporate pheromone by reducing the pheromone on all paths by

Algorithm 5 ACO for the Longest Induced Path Problem

```
1: procedure ACO( $G$ )
2:    $tasks \leftarrow \{v \in V : \deg(v) > 2\}$ 
3:   Initialize pheromone matrix  $\tau$  with  $\tau_{\min}$  on all edges
4:    $P_{\text{best}} \leftarrow \emptyset$ 
5:    $stagnation \leftarrow 0$ ;  $restarts \leftarrow 0$ ;  $tabu \leftarrow \emptyset$ 
6:   while time limit not exceeded and  $stagnation \leq maxStagnation$  do
7:      $iterPaths \leftarrow \emptyset$ 
8:     for each  $v \in tasks$  such that  $v \notin P_{\text{best}}$ , in parallel do
9:        $P \leftarrow \text{PHEROMONEGUIDEDDFS}(v)$ 
10:       $iterPaths \leftarrow iterPaths \cup \{P\}$ 
11:    end for
12:     $P_{\text{iter}} \leftarrow \text{longest path in } iterPaths$ 
13:    if  $|P_{\text{iter}}| > |P_{\text{best}}|$  then
14:       $P_{\text{best}} \leftarrow P_{\text{iter}}$ 
15:       $stagnation \leftarrow 0$ ;  $restarts \leftarrow 0$ ;  $\rho \leftarrow \rho_{\min}$ 
16:    else
17:       $stagnation \leftarrow stagnation + 1$ 
18:      if  $stagnation \bmod restartInterval = 0$  then
19:        Reset all pheromone to  $\tau_{\min}$ 
20:         $tabu \leftarrow tabu \cup \{P_{\text{best}}\}$ 
21:         $restarts \leftarrow restarts + 1$ 
22:         $dfsCount \leftarrow 2 + \lfloor \log_{10}(\max(1, restarts)) \rfloor$ 
23:        continue
24:      else if  $stagnation \bmod stagnationThreshold = 0$  then
25:         $\rho \leftarrow \min(\rho + \Delta\rho, \rho_{\max})$ 
26:      end if
27:    end if
28:    EVAPORATEPHEROMONE
29:    if global improvement found then
30:      DEPOSITPHEROMONE( $P_{\text{best}}$ , 2)
31:    else
32:      Deposit on top- $k$  non-tabu iteration paths with factor 1
33:    end if
34:  end while
35:  return  $P_{\text{best}}$ 
36: end procedure
```

an amount. For depositing pheromone, if we've found a global improvement then we deposit pheromone only on that path with double the value, otherwise we deposit on all top-k paths (excluding those in the tabu list) with a factor of 1.

2.1.2 Path construction via pheromone-guided DFS

The core of the ACO algorithm is the path construction procedure. The standard ACO algorithm performs just a forward walk, in our case it was modified to be a DFS pheromone-guided search with backtracking. This implementation is detailed in algorithm 6 below.

Algorithm 6 Pheromone-Guided DFS

```

1: procedure PHEROMONEGUIDEDDFS(start)
2:    $P_{\text{best}} \leftarrow \text{DFS}(\textit{start})$ 
3:   for  $i = 1$  to  $\textit{dfsCount} - 1$  do
4:      $P' \leftarrow \text{DFS}(\text{last vertex of } P_{\text{best}})$ 
5:     if  $|P'| > |P_{\text{best}}|$  then
6:        $P_{\text{best}} \leftarrow P'$ 
7:     end if
8:   end for
9:   return  $P_{\text{best}}$ 
10: end procedure

11: procedure DFS(start)
12:    $P_{\text{work}} \leftarrow [\textit{start}]; P_{\text{best}} \leftarrow \emptyset$ 
13:    $\textit{visited}[\textit{start}] \leftarrow \textit{true}$ 
14:   UPDATEINVALIDCOUNT(start, +1)
15:   while  $|P_{\text{work}}| > 0$  do
16:      $v \leftarrow \text{last vertex of } P_{\text{work}}$ 
17:      $\textit{candidates} \leftarrow \{u \in N(v) : \neg \textit{visited}[u] \wedge \textit{invalidCount}[u] = 1\}$ 
18:     if  $\textit{candidates} \neq \emptyset$  then
19:        $u \leftarrow \text{SELECTCANDIDATE}(v, \textit{candidates})$ 
20:        $\textit{visited}[u] \leftarrow \textit{true}$ 
21:        $P_{\text{work}} \leftarrow P_{\text{work}} \oplus \{u\}$ 
22:       UPDATEINVALIDCOUNT(u, +1)
23:     else
24:       if and  $|P_{\text{work}}| > |P_{\text{best}}|$  then
25:          $P_{\text{best}} \leftarrow P_{\text{work}}$ 
26:       end if
27:        $\textit{removed} \leftarrow \text{last vertex of } P_{\text{work}}$ 
28:       Remove removed from  $P_{\text{work}}$ 
29:       UPDATEINVALIDCOUNT(removed, -1)
30:     end if
31:   end while
32:   return  $P_{\text{best}}$ 
33: end procedure

```

The outer procedure (lines 1-10) called *PheromoneGuidedDFS* is responsible of calling the inner DFS method. It first calls DFS for the starting node (the task sent by the previous method). Then, using a loop it calls DFS *dfsCount* times from the last vertex of the best path found by the previous DFS. This has the goal to explore bidirectionally the graph as a single DFS from a fixed start can only explore one direction of the path. Starting a second DFS from the final endpoint lets the algorithm search for extensions in the opposite direction. The method returns the longest induced path found from all the DFS, the initial value of *dfsCount* is set to 2 and is increased adaptively after restarts of the algorithm.

The inner DFS procedure (lines 11-33) is the standard DFS modified to produce only induced paths and to select probabilistically pick the next vertex based on the pheromone values on the edges. Lines 12-14 initialize the variables, *UpdateInvalidCount* method is used to update the invalid array for the neighbors of the given vertex. This array is used to determine how many neighbors this vertex has within the induced path. If the vertex has only one neighbor (*invalidCount* = 1) then its single neighbor is the current vertex which means we can add this vertex to the induced path without forming a cycle.

Lines 15-30 is the DFS exploration loop, which ends when we no longer can expand the path. We take the last vertex in the path in line 16 and get the valid neighbors of its vertex. A neighbor is valid if it hasn't been visited yet (when we backtrack we set this vertex to visited to avoid using it again) and if its *invalidCount* is set to 1 (its single neighbor is the current vertex). The if in lines 18 check if we have any candidates and if we have any then we pick one probabilistically using the pheromones on the edges. Lines 20-22 adds the new vertex to the path and updates the invalid count array for its neighbors. Lines 24-29 handle the case when we cannot extend the path any further. It stores the path if it is the longest found so far and removes the last vertex from the path. The method ends by returning the longest path found.

2.1.3 Candidate selection

When we have multiple candidates to expand the path with, the algorithm selects the next vertex probabilistically based on the values of the pheromones. For each candidate u , its selection weight is equal to:

$$w(u) = \tau(v, u)^\alpha$$

where $\tau(v, u)$ is the pheromone level on the directed edge from v to u , and α is the pheromone influence parameter (this is set to 1 by default). Using this value, the probability of selecting candidate u is:

$$p(u) = \frac{w(u)}{\sum_{u' \in \text{candidates}} w(u')}$$

The selection method is roulette wheel sampling which draws a random number uniformly from $[0, 1)$ and the candidate whose cumulative probability range contains the drawn number is selected.

The algorithm also has a disabled heuristic component based on *forward checking* which is the number of unvisited neighbors of u that are not yet invalidated. When enabled, the selection weight becomes $w(u) = \tau(v, u)^\alpha \cdot h(u)^\beta$, where $h(u)$ is the forward check value and β is the heuristic influence parameter. Through comprehensive testing, it was discovered that the heuristic component turned out to be counterproductive as it biases the search toward high-degree vertices, which often leads to dead ends in dense regions of the graph. The default configuration disables it ($\beta = 0$) and keeps the candidate selection to rely purely on pheromone.

2.1.4 Pheromone model

The pheromone model uses *directed edge pheromones*, the pheromone value $\tau(v, u)$ is stored independently from $\tau(u, v)$ which means the direction of the traversal matters and we treat the edges as directional edges. Moving from vertex v to vertex u is different than moving from vertex u to v as the set of available vertices depends on the entire path built so far. This means that the potential path that can be built from moving in that direction is also different. Take for example a dead end: moving towards it could have a low pheromone value but moving outside of it could have a higher pheromone as it has a higher potential to form a longer induced path. Symmetric pheromone would lose the information about which direction of traversal was successful as it would conflate these two situations.

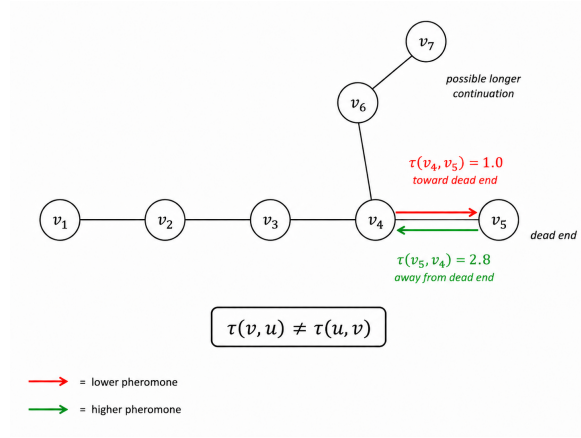


Figure 2.2: Explanation of directed edge pheromones

We bound the pheromone values between the interval $[\tau_{\min}, \tau_{\max}]$ based on the MAX-MIN Ant System (MMAS) framework of Stützle and Hoos [?]. These bounds are used to prevent the algorithm from becoming a greedy algorithm over time choosing only the best edges and preventing any further exploration. The bound values are set to $\tau_{\min} = 1.0$ and $\tau_{\max} = 3|V|$.

2.1.5 Pheromone operations

Initialization. The values of pheromones on each directed edge are initialized to $\tau_{\min}$, so that in the first iteration all edges have an equal chance to be selected.

Evaporation. After each iteration, we reduce all pheromone values by a factor of $(1 - \rho)$:

$$\tau(v, u) \leftarrow (1 - \rho) \cdot \tau(v, u), \quad \forall (v, u) \in E$$

where ρ is called the evaporation rate. This lets the algorithm adapt over time by gradually forgetting information from old iterations. The evaporation rate is adaptive as it starts at $\rho_{\min} = 0.1$ and increases by $\Delta\rho = 0.1$ every *stagnationThreshold* non-improving iterations, up to $\rho_{\max} = 0.9$. When we find a global improvement or when we restart the algorithm, ρ is reset to $\rho_{\min}$. Low evaporation makes the algorithm behave more greedy as learned information is used more often when the algorithm is making progress while high evaporation forces the algorithm to try new paths when it is stuck.

Deposit. Pheromone is deposited using a scheme dependent on the position. Given a path $P = [v_0, v_1, \dots, v_{k-1}]$ and a reinforcement factor f , the deposit is:

$$\tau(v_i, v_{i+1}) \leftarrow \tau(v_i, v_{i+1}) + f \cdot (k - i - 1), \quad i = 0, 1, \dots, k - 2 \quad (\text{forward})$$

$$\tau(v_i, v_{i-1}) \leftarrow \tau(v_i, v_{i-1}) + f \cdot (k - i - 1), \quad i = k - 1, k - 2, \dots, 1 \quad (\text{backward})$$

The goal of this is to deposit pheromone based on the potential length of the induced path we could get by choosing this edge. In the forward pass, earlier edges get more pheromones as we are at the start of the path and the length of the induced path we can get is the highest at this point (edge $v_0 \rightarrow v_1$ receives $f(k - 1)$, while edge $v_{k-2} \rightarrow v_{k-1}$ receives f). The backward pass mirrors this as edge $v_{k-1} \rightarrow v_{k-2}$ receives $f(k - 1)$, while edge $v_1 \rightarrow v_0$ receives f . This leads to the middle of the path receiving the most total pheromone. For a path of 5 vertices, the combined deposits on the edge between v_1 and v_2 are $3f + 2f = 5f$ (forward $v_1 \rightarrow v_2$ plus backward $v_2 \rightarrow v_1$), while the endpoints receive $4f + 1f = 5f$ each but only in one direction. Storing the direction in the pheromone matrix adds more information for the algorithm to learn while it encourages ants to follow the core of good paths.

Two strategies for depositing pheromones are used depending on whether we have found a global improvement:

- **Global improvement:** we deposit pheromone only on the new global best path with a reinforcement factor of $f = 2$, strongly reinforcing the best solution found.
- **No improvement:** we deposit pheromone on the top k distinct paths found in the current iteration (default $k = 5$), sorted by decreasing length, with a factor of $f = 1$. Paths in the tabu list are excluded. Depositing on multiple paths rather than just the iteration best increases the exploration of the algorithm as we are within a local maximum and we should try other paths as well.

2.1.6 Stagnation management

To prevent the algorithm from getting stuck in a local maximum, three mechanisms are used to handle stagnation:

Adaptive evaporation. As described above, the evaporation rate is gradually increased if we haven't found an improvement the last iterations. This speeds up the

forgetting of current pheromone trails without fully discarding learned information allowing the algorithm to learn new trails more easily.

Pheromone restarts. If we haven't found an improvement in the last *restartInterval* iterations (default: 100) then we perform a full pheromone restart: all pheromone values are reset to $\tau_{\min}$, the evaporation rate is restored to $\rho_{\min}$, and we increase the DFS count. This gives the algorithm another chance to find the global maximum while retaining the current longest induced path found.

Tabu list. When a restart occurs, the current best path is added to a list called tabu. During subsequent pheromone deposits, we exclude any path matching one in the tabu list. This prevents the algorithm from rediscovering and reinforcing the same solution after a restart. The tabu list uses a hash-based set of vertex arrays for efficient lookup.

These three mechanisms form a hierarchy based on the amount of stagnation: minor stagnation triggers evaporation rate increases, moderate stagnation triggers restarts, and the tabu list ensures that restarts lead to genuinely different search regions. We terminate the algorithm after *maxStagnation* = 2000 consecutive iterations where no improvement was found since even restarts failed to produce improvements.

2.1.7 Parallelism

The algorithm uses parallelism at the task level. Each DFS task is submitted to a thread pool and executed in parallel. Each DFS task reads the shared pheromone matrix but do not write to it during the path construction. As we only read from this matrix and no writes are made, this ensures it doesn't need any synchronization. Pheromone updates (evaporation and deposit) are performed after the tasks are executed and they are performed sequentially between iterations. The thread pool size used is the number of available CPU cores.

The table below 2.1 summarizes the parameters of the ACO algorithm and their default values.

Parameter	Default	Description
$\tau_{\min}$	1.0	Minimum pheromone level
$\tau_{\max}$	$3 V $	Maximum pheromone level
$\rho_{\min}$	0.1	Initial / minimum evaporation rate
$\rho_{\max}$	0.9	Maximum evaporation rate
$\Delta\rho$	0.1	Evaporation rate increase step
α	1.0	Pheromone influence exponent
β	0.0	Heuristic influence exponent (disabled)
<i>dfsCount</i>	2	DFS runs per task (adaptive)
<i>maxDepositPaths</i>	5	Paths to deposit on per iteration
<i>maxStagnation</i>	2 000	Max non-improving iterations
<i>restartInterval</i>	100	Iterations between restarts
<i>stagnationThreshold</i>	10	Iterations between evaporation increases
time limit	15 min	Wall-clock time limit

Table 2.1: Parameters of the ACO algorithm

2.2 Genetic Algorithm

This section is dedicated to the genetic algorithm proposed for this problem. It is a complete redesign of the genetic algorithm I proposed in my bachelor thesis. The earlier GA relied on a boolean-array representation of each individual which caused the algorithm to have a higher time complexity than needed for many of its operations. The individuals after crossover required expensive repair using a modified heuristic that took a significant amount of time. The mutation operator of adding and removing vertices, while the most useful, made only small changes, which caused the population to converge prematurely, leading the algorithm to get stuck in a local maximum.

The new genetic algorithm addresses these issues by replacing the representation with an ordered vertex array. This improves the efficiency of the operators significantly as we no longer have to traverse the entire individual array for most operators (which was equal to the number of nodes in the graph). The mutation operator was extended to remove then add more vertices, while the crossover was changed to combine the vertex sets of both parents. Both of these operators use a modified DFS which has the goal of finding maximal induced paths locally. The mutation operator uses this

to check for any further vertices that can be added. Crossover uses the modified DFS to check the longest induced path within the union set of the parents, then within the entire graph using the previously found path as the starting path. The algorithm also injects fresh individuals over time to maintain diversity and prevent stagnation.

2.2.1 Algorithm overview

Algorithm 7 presents the main flow of the genetic algorithm. The population goes through multiple iterations of genetic operators: selection, crossover and mutation. Fresh injection is also used at each step to enhance the population diversity. The algorithm runs until the time limit is reached or till we have reached $maxGen$ iterations. The $maxGen$ variable is increased every time we have found an improvement to allow the algorithm to continue if it keeps finding better solutions and to stop otherwise.

Algorithm 7 Genetic Algorithm for the Longest Induced Path Problem

```

1: procedure GENETICALGORITHM( $G$ )
2:    $population \leftarrow \text{INITPOPULATION}(popSize)$  ▷ parallel biDFS
3:    $P_{best} \leftarrow \text{longest path in } population$ 
4:    $genSinceImprove \leftarrow 0$ 
5:    $maxGen \leftarrow \text{startingMaxGenerations}$ 
6:   while time limit not exceeded and  $gen < maxGen$  do
7:      $population \leftarrow \text{SELECT}(population)$ 
8:      $\text{CROSSOVER}(population)$ 
9:      $\text{MUTATION}(population)$ 
10:     $\text{INJECTFRESH}(population)$ 
11:    if  $population$  contains a path longer than  $P_{best}$  then
12:       $P_{best} \leftarrow \text{new best}; genSinceImprove \leftarrow 0$ 
13:       $maxGen \leftarrow \max(maxGen, gen + \text{improvementPatience})$ 
14:    else
15:       $genSinceImprove \leftarrow genSinceImprove + 1$ 
16:    end if
17:    if  $genSinceImprove \geq \text{stagnationLimit}$  then
18:      Keep top  $eliteCount$  individuals
19:      Replace  $restartRatio$  fraction with fresh biDFS individuals
20:       $genSinceImprove \leftarrow 0$ 
21:    end if
22:  end while
23:  return  $P_{best}$ 
24: end procedure

```

Lines 2-5 initialize the variables: the population is initialized by running the DFS multiple times providing a good start to the algorithm, the longest individual is saved

in P_{best} , and the $genSinceImprove$ and $maxGen$ are the variables used to determine when to stop the algorithm, initialized with 0 and $startingMaxGenerations$.

Lines 6-22 represent the main loop of the algorithm where we apply the genetic operators every iteration: selection, crossover and mutation. We also inject fresh individuals into the population using the modified DFS to prevent stagnation. Lines 11-16 save the best individual in the population and handle the stopping logic. We increase the $maxGen$ variable by $\max(maxGen, gen + improvementPatience)$ which prevents the algorithm from stopping if it found an improvement in the last $improvementPatience$ iterations. Lines 17-21 is the stagnation logic, it replaces $restartRatio$ fraction of the population (set to 60%) with fresh individuals if the algorithm has stagnated for many iterations.

2.2.2 Individual representation

Each individual is represented by an ordered array of vertex indices $[v_0, v_1, \dots, v_{k-1}]$ which form a valid induced path in the graph. In this case, v_0 is the starting vertex in the path and v_{k-1} represents the last vertex. Consecutive vertices in the arrays are adjacent in the graph and there are no edges connecting non-consecutive vertices to maintain the induced path property. The fitness of an individual represents its length (the number of vertices in the path) and mutation and the DFS were designed to run efficiently using this representation as getting the last vertex or starting vertex and extending it can be done in constant time ($O(1)$).

Example: Representation of an Individual for the Longest Induced Path Problem

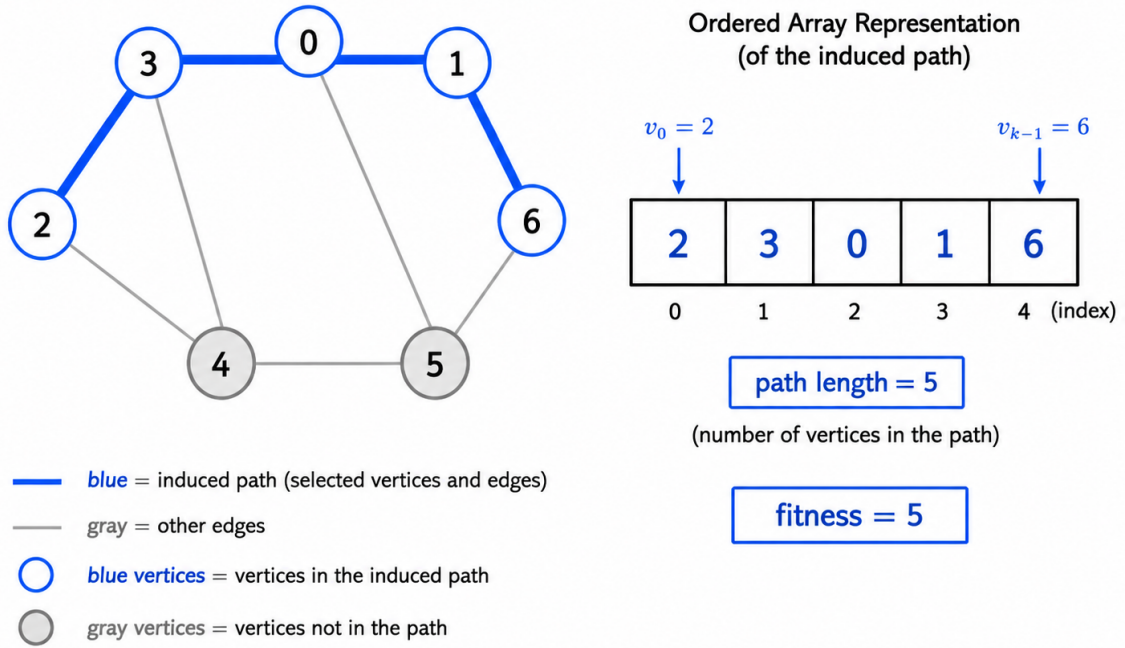


Figure 2.3: Individual representation

There is also another representation that can be used in this problem where each position corresponding to a vertex is a true/false value indicating the membership in the path. This representation loses the ordering information causing the operators and DFS to run in a higher time complexity as it first requires recovering the order of vertices from the boolean array. After testing, it was decided to keep the ordered array representation as it resulted in the algorithm being significantly faster.

2.2.3 Population initialization

The initial population is created using the bidirectional DFS algorithm (Algorithm 9). Each individual is created by selecting a random starting vertex and running more DFS explorations to produce a locally maximal induced path. These executions are started in parallel and the number of individuals generated is set to 200.

2.2.4 Core DFS with backtracking

The basic building block of the genetic algorithm is a randomized DFS procedure. This algorithm, called *coreDFS*, constructs an induced path using depth-first search modified to select valid candidates uniformly at random. The algorithm also backtracks to try other paths with a maximum amount of paths to prevent it from taking a significant amount of time. It is used in population initialization, mutation, crossover and path extension.

Algorithm 8 Core DFS with Backtracking

```

1: procedure COREDFS(start, visited, adjCount, filter)
2:    $P_{\text{work}} \leftarrow [start]; P_{\text{best}} \leftarrow [start]$ 
3:    $paths \leftarrow 0; lastImprov \leftarrow 0; forward \leftarrow true$ 
4:   while  $|P_{\text{work}}| > 0$  and  $paths - lastImprov \leq maxPaths$  do
5:      $v \leftarrow \text{last vertex of } P_{\text{work}}$ 
6:     if forward then
7:        $candidates \leftarrow \{u \in N(v) : \neg visited[u] \wedge adjCount[u] = 1 \wedge (filter =$ 
      null  $\vee filter[u])\}$ 
8:     end if
9:     if  $|candidates| > 0$  then
10:       $u \leftarrow \text{random vertex from } candidates$ 
11:      Remove u from candidates
12:       $P_{\text{work}} \leftarrow P_{\text{work}} \oplus \{u\}$ 
13:       $visited[u] \leftarrow true$ 
14:       $adjCount[w] \leftarrow adjCount[w] + 1, \forall w \in N(u)$ 
15:      forward  $\leftarrow true$ 
16:     else
17:       if forward then
18:          $paths \leftarrow paths + 1$ 
19:         if  $|P_{\text{work}}| > |P_{\text{best}}|$  then
20:            $P_{\text{best}} \leftarrow \text{copy of } P_{\text{work}}$ 
21:            $lastImprov \leftarrow paths$ 
22:         end if
23:       end if
24:        $removed \leftarrow \text{last vertex of } P_{\text{work}}$ 
25:       Remove removed from  $P_{\text{work}}$ 
26:        $visited[removed] \leftarrow false$ 
27:        $adjCount[w] \leftarrow adjCount[w] - 1, \forall w \in N(removed)$ 
28:       forward  $\leftarrow false$ 
29:     end if
30:   end while
31:   return  $P_{\text{best}}$ 
32: end procedure

```

The method takes as input the starting vertex and the *visited*, *adjCount* and *filter* arrays. The *visited* and *adjCount* arrays are used to give as parameter an existing path

for the DFS to attempt to extend it further. This is used for example in the mutation operator to further expand any individual. The *adjCount* ensures we keep the induced path property as it stores the amount of neighbors in the current path for each vertex. We pick vertices only if they have *adjCount* set to 1 meaning only one adjacent vertex within the path, this vertex being the current vertex. This prevents creating any cycles and keeps the induced path property.

Lines 4-30 contain the DFS loop which runs till the current path is not empty or till a maximum amount of paths generated. Lines 5 sets v to the last vertex of the current path P_{work} . Then, we check the value of the *forward* variable which is used to determine if we are generating paths (going forward) or backtracking (going backwards). If it is set to true, then we did not backtrack so we generate the candidates. Each candidate must not be visited (part of the path already), must have only one neighbor within the path (*invCount*[u] is set to 1) and the filter for this vertex must be set to true if this parameter is given. This array called *filter* is used in the crossover operator to limit the search only inside a subgraph. If we have candidates then we pick one at random, remove it from the candidates and add it to the path. Otherwise, if *forward* is true then we increment the number of paths and save the current one if it is longer than the best one found so far. After this, lines 24-28, prepare the variables for backtracking by removing the last vertex from the path, decrementing the value of *invCount* for its neighbors and setting *forward* to false. The method returns the longest path found.

Bidirectional exploration is handled at a higher level through the *biDFS* and *extend* procedures which are described next.

2.2.5 Bidirectional DFS

The *biDFS* procedure chains multiple DFS runs for bidirectional exploration:

Algorithm 9 Bidirectional DFS

```

1: procedure BiDFS(start)
2:    $P \leftarrow \text{FRESHDFS}(\text{start})$ 
3:   for  $i = 1$  to  $\text{dfsCount} - 1$  do
4:      $P' \leftarrow \text{FRESHDFS}(\text{last vertex of } P)$ 
5:     if  $|P'| > |P|$  then
6:        $P \leftarrow P'$ 
7:     end if
8:   end for
9:   return  $P$ 
10: end procedure

```

The procedure starts by first running a DFS from the starting vertex which creates the initial induced path found. In order to explore in the other direction, the loop within lines 3-8 call DFS again but this time using the last vertex of the path found. Essentially, we are calling DFS from an endpoint vertex which will allow us to explore the opposite direction. This is based upon the diameter approximation algorithm which runs DFS twice to find the diameter of a graph. The *FreshDFS* method just calls *DFS* after creating the *visited* and *invCount* arrays using the vertex given so each run is independent.

2.2.6 Path extension

The path extension method is used to grow an individual from both endpoints using DFS. This is used after the mutation and crossover operator as they can possibly produce a shortened path and we want to ensure we are working with the longest paths we can find locally.

Algorithm 10 Bidirectional Path Extension

```

1: procedure EXTEND(seed)
2:   if seed =  $\emptyset$  then
3:     return BiDFS(random vertex)
4:   end if
5:   Initialize inPath and invCount from seed
6:   fwdPath  $\leftarrow$  COREDFS(last vertex of seed, inPath, invCount, null)
7:   Update inPath and invCount with vertices from fwdPath
8:   bwdPath  $\leftarrow$  COREDFS(first vertex of seed, inPath, invCount, null)
9:   return reverse(bwdPath) || seed || fwdPath
10: end procedure

```

The procedure takes a seed path which is an individual (the output of mutation or crossover) and extends it in both directions using DFS. Lines 2-4 handle the case where we receive an empty seed so we just run BiDFS method with a random vertex in the graph. Lines 5-9 contain the logic of the method, we run a DFS from the last vertex from the seed which is the forward path, then update the *inPath* and *invCount* variables given as parameters to the DFS and run the DFS from the starting vertex so we get the backwards path. The variable returns the backwards path concatenated with the initial path concatenated with the forward path obtaining a new induced path which has been extended from both directions.

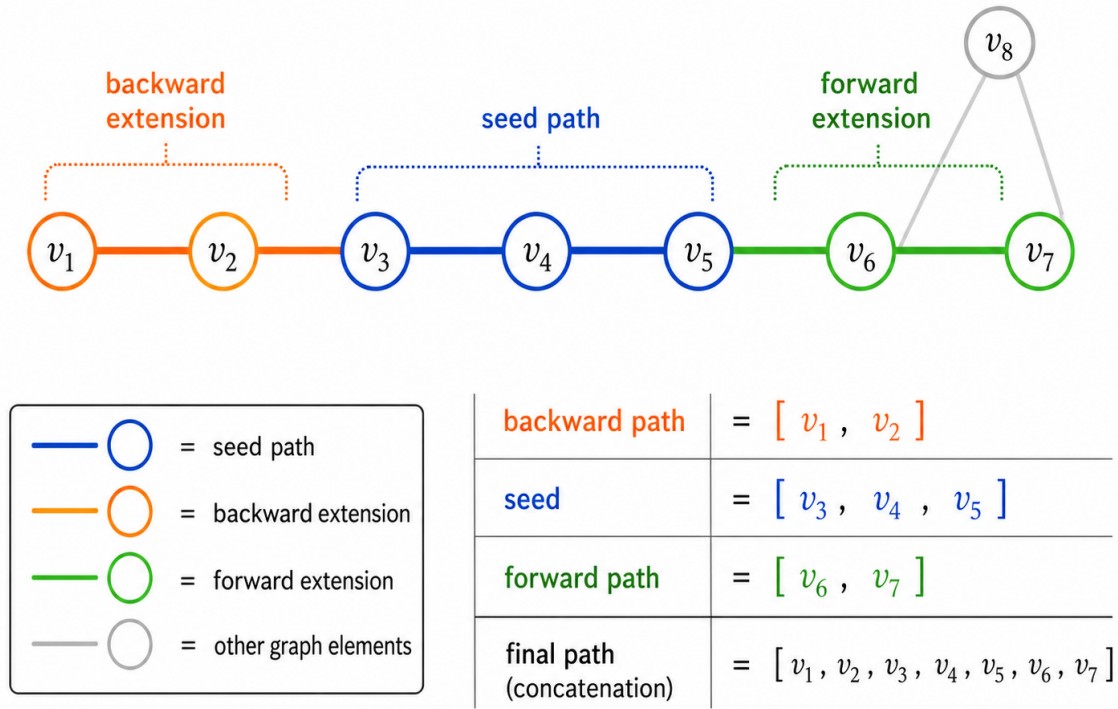


Figure 2.4: Genetic path extension

2.2.7 Mutation operator

The mutation operator takes an individual and modifies it by trimming vertices from both endpoints and then re-extending the result:

Algorithm 11 Trim-and-Extend Mutation

```

1: procedure MUTATE( $path$ )
2:    $maxTrim \leftarrow \max(2, |path|/maxTrimDivisor)$ 
3:    $trimL \leftarrow \text{random integer in } [0, maxTrim)$ 
4:    $trimR \leftarrow \text{random integer in } [0, maxTrim)$ 
5:   if  $trimL + trimR \geq |path| - 1$  then
6:     return BDFS(random vertex)
7:   end if
8:    $middle \leftarrow path[trimL \dots |path| - trimR - 1]$ 
9:   return EXTEND( $middle$ )
10: end procedure

```

Lines 2-4 pick how many vertices to trim from each side of the path at random. $maxTrimDivisor$ is a parameter of the genetic algorithm deciding how much of a percentage to trim at most from each side. After testing, the best value was found to be

4. Lines 5-7 handle the case where the path is very short where by trimming it we would remain with no vertices. In this scenario, we replace the individual with a run of *BiDFS* method that will produce a new path. Lines 8-9 call the *extend* method for what remains of the individual after removing vertices from the left and right sides.

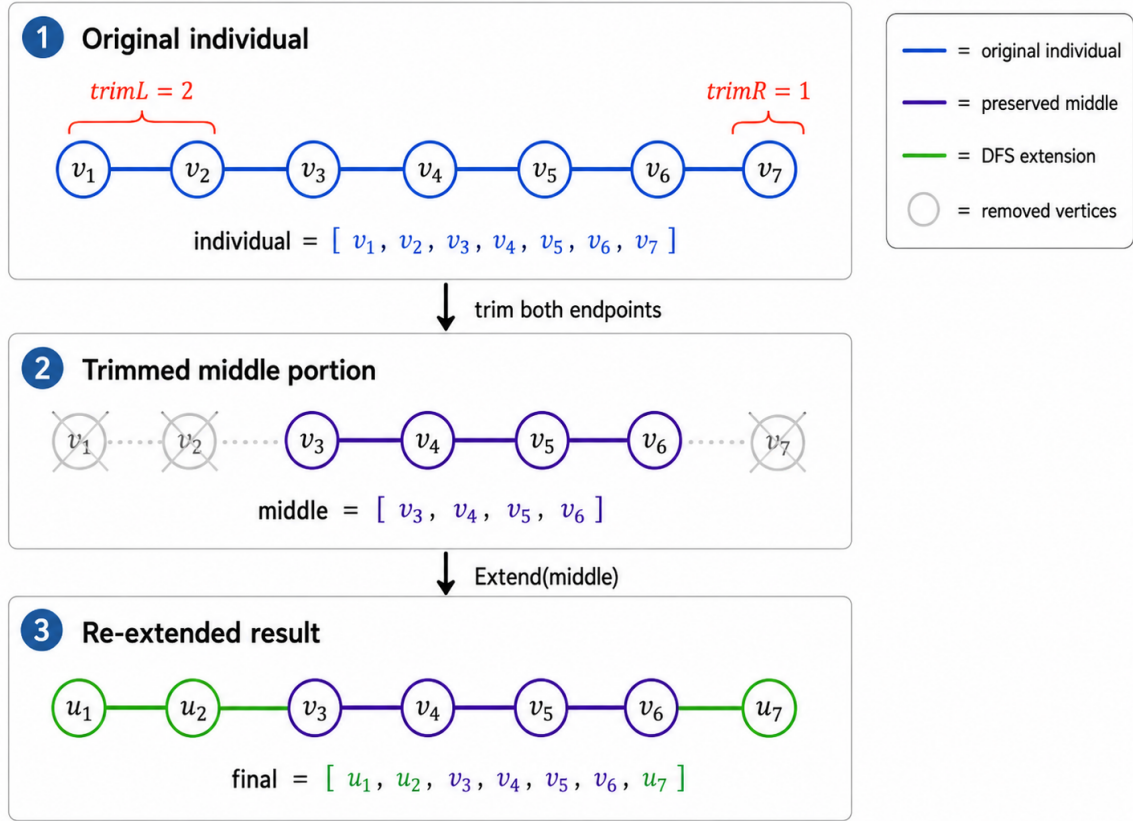


Figure 2.5: Mutation Operator

In the example above, $[v_1, v_2]$ vertices from the left and $[v_7]$ from the right has been trimmed then replaced by $[u_1, u_2]$ and $[u_7]$ respectively. The purpose of this operator is to keep the middle of each individual which contains the best vertices which are contained by most induced paths within the graphs. By constantly removing and adding vertices in each direction, we do a localized search with the goal of finding longer induced paths. The individual obtained in this way is added to the population and does not replace the individual given to be mutated.

2.2.8 Crossover operator

The crossover operator combines two individuals by taking the union of their vertex sets and finding an induced path within this union. Afterwards, we extend the

induced path found with vertices from the entire graph.

Algorithm 12 Union-Based Crossover

```

1: procedure CROSSOVER( $p_1, p_2$ )
2:    $union \leftarrow \{v : v \in p_1 \vee v \in p_2\}$ 
3:   for each  $v$  such that  $v \in p_1 \wedge v \in p_2$  do
4:     if  $random < crossoverRemoveRate$  then
5:        $union \leftarrow union \setminus \{v\}$ 
6:     end if
7:   end for
8:    $start \leftarrow$  first vertex of  $p_1$  if in  $union$ , else first of  $p_2$  if in  $union$ , else random
9:   Initialize  $visited$  and  $invCount$  from  $start$ 
10:   $subPath \leftarrow COREDFS(start, visited, invCount, union)$ 
11:  return EXTEND( $subPath$ )
12: end procedure

```

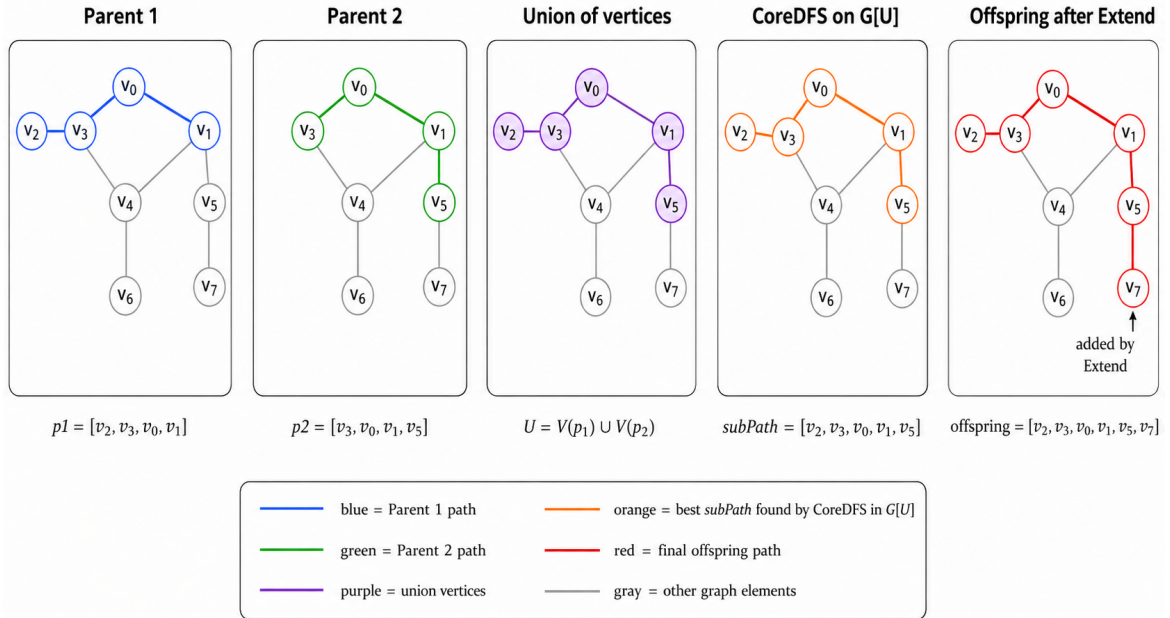


Figure 2.6: Crossover

The crossover begins by creating a union of the vertices from both parents. Then, for each vertex in the union, we have a probability of removing a vertex (set to 0.5). By creating a union of two induced paths, the path we get most likely isn't an induced path anymore. Furthermore, by removing even vertices from it (to increase the exploration factor), the chances are even higher. This is addressed by running DFS inside the graph induced by this vertex set then we extend the induced path we obtained using the vertices from the entire graph.

This operators role is to diversify the population by radically changing individu-

als by combining them. The individual obtained in this way are added to the population and he does not replace the two individuals given.

2.2.9 Selection operator

The selection operator has the goal of generating the next generation from the current population. The individuals with the highest fitness (length of the induced path) have higher chances of surviving and making it to the next generation to go through the mutation and crossover again. Two selection operators have been tested, with the default one being the tournament selection:

Tournament selection (default). For each slot in the new population (beyond the elite), two individuals are picked at random and the one with the highest fitness takes the slot. This selection is time efficient taking only $O(popSize)$ time but provides moderate selection pressure and individuals with lower fitness have no chance of surviving. For example, the individual with the lowest fitness always loses and is never preserved in the next generation.

Roulette wheel selection. In this selection, we assign to each individual a probability directly proportional to its fitness (path length). Individuals are then sampled with replacement to fill the new population. This method gives smoother selection pressure than tournament selection but is significantly slower.

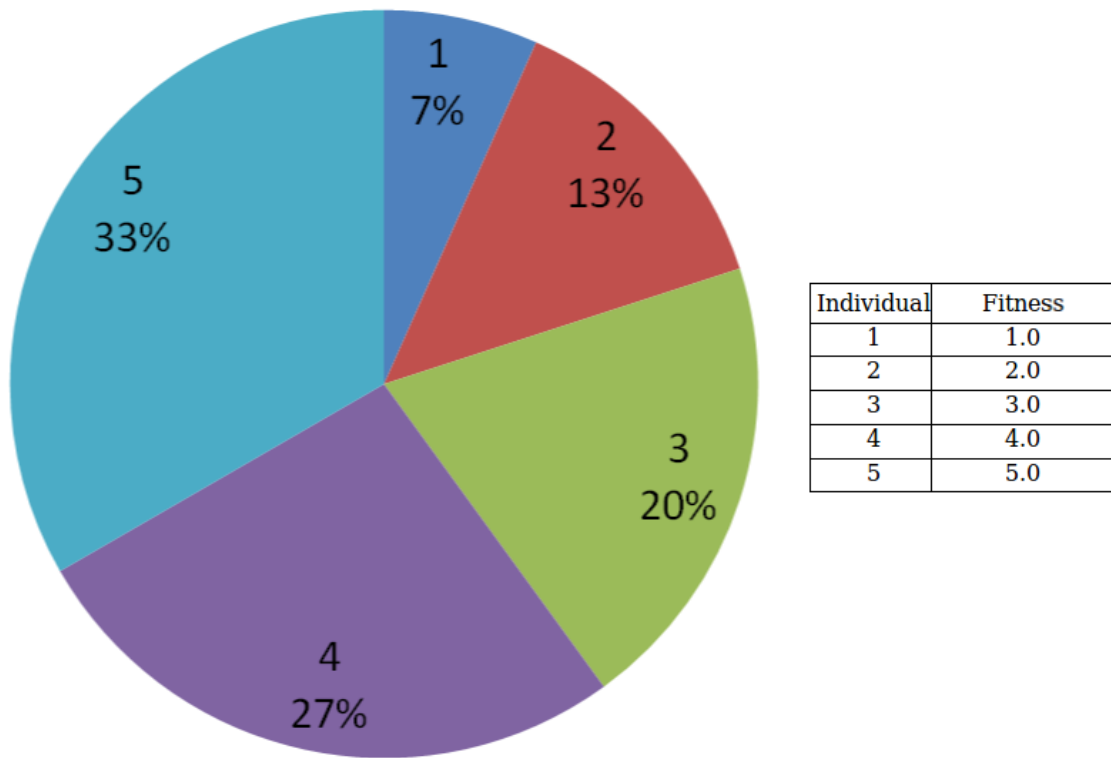


Figure 2.7: Roulette Wheel Selection

Both methods copy the best *eliteCount* individuals (the default is $\max(2, \text{popSize}/20)$, which gives 10 for the default population size of 200) directly to the next generation. This is called *elitism* and ensures that the best solutions are never lost to the stochastic selection process.

2.2.10 Fresh individual injection

To prevent the population from converging to a single solution prematurely, the algorithm injects freshly generated individuals at each generation. For each of the *popSize* slots, a new individual is generated via biDFS with probability *freshRate* (default: 0.1), creating roughly $\text{popSize} \times \text{freshRate}$ new individuals per generation (about 20 for the default population size of 200). These fresh individuals are added alongside the results of crossover and mutation, temporarily increasing the population size until the next selection step reduces it back to *popSize*. The injection of new individuals help balance the elitism count which tends to create many copies of the best individuals in the generation.

2.2.11 Stagnation restart

When we detect that the algorithm has reached a local maximum by seeing no improvements in the last *stagnationLimit* consecutive generations (default: 500), the algorithm performs a partial population restart. The top *eliteCount* individuals are preserved, and a fraction *restartRatio* (the default is 0.6) of the remaining population is replaced with new individuals generated using *biDFS* method. This increases temporarily the exploration part of the genetic algorithm while preserving the best individuals found so far.

The table below 2.2 contains the parameters of the genetic algorithm and their default values.

Parameter	Default	Description
<i>popSize</i>	200	Population size
<i>eliteCount</i>	$\max(2, popSize/20)$	Number of elite individuals preserved
<i>mutationRate</i>	0.3	Probability of mutating a non-elite individual
<i>crossoverRate</i>	0.2	Probability of crossover per pair
<i>freshRate</i>	0.1	Per-slot probability of fresh injection
<i>crossoverRemoveRate</i>	0.5	Probability of removing shared vertices in crossover
<i>maxTrimDivisor</i>	4	Max trim per side = $ path /maxTrimDivisor$
<i>dfsCount</i>	2	DFS runs in biDFS
<i>maxPaths</i>	10	Max non-improving paths before DFS stops
<i>stagnationLimit</i>	500	Generations before stagnation restart
<i>restartRatio</i>	0.6	Fraction of population replaced on restart
time limit	2 min	Wall-clock time limit

Table 2.2: Parameters of the genetic algorithm

Chapter 3

Computational Experiments

This chapter contains the computational experiments for the algorithms proposed in comparison with the current state of the art algorithms. To be precise, I will compare the ACO and genetic algorithm proposed with the branch-and-cut algorithms CEC and CUT from Marzo et al. [2] and the HLIPP heuristic from Marzo and Ribeiro [1]. All algorithms run on the same hardware with the same amount of time limit and with the same resources. The exact algorithms failed to produce any significant results on larger graphs to be included in the comparison. Their results are still present in the papers referenced throughout this thesis as most of the instances are shared.

3.1 Test Instances

The test instances used are real world network instances from Marzo and Ribeiro [1] which are the standard in the literature around this problem. They range from social networks with 33 networks to the yeast protein interaction network which has 2361 vertices and 6646 edges. We know the optimal values for most of these instances using exact algorithms or from prior work. The yeast instance has no known optimal value as no exact algorithm has solved it within practical time limits. I've also generated a synthetic instance with 5000 vertices and over 1.300.000 edges with a known lower bound on the optimal solution using the generator described in the next section. This instance is meant as a very hard instance to solve for any algorithm and is used to compare the algorithms on dense graphs.

3.1.1 Instance generator

For real-world large instances which are hard to solve, the optimal solution is unknown so there is no way to measure how close each algorithm gets to finding the optimal solution. To help with this, I've built an instance generator capable of generating hard to solve instances with a known lower bound for the optimal path. While there exists a chance for an induced path to exist that is longer than the known lower bound, the difference would be at most a few vertices. The algorithm consists of a planted path of a fixed size then adding new vertices and edges in such a way to complicate any algorithm trying to find it. At the end, the algorithm does a double check to check that the path is still induced.

The generator has four steps:

1. **Planted path.** A simple path of length L is created using vertices $\{0, 1, \dots, L - 1\}$ with edges connecting consecutive vertices.
2. **Noise vertices.** Create $N - L$ vertices as noise then we connect them to a percentage of the vertices in the real path (depending on the difficulty setting, from 20-35%). Since each noise vertex touches many path vertices, including it in an induced path forces the exclusion of more path vertices than it contributes which guarantees that it is always a net loss.
3. **Clique structure.** Noise vertices are organized into small cliques of size 3, which limits any induced path through the noise subgraph to at most 2 vertices per clique. We also add edges between cliques with a low probability (10%) to prevent the noise structure from being easily separable.
4. **Trap structures.** Add short paths branching off near the endpoints of the planted path. These paths are traps which look like promising extensions but lead into the clique-structured noise region where further extension is quickly blocked.

The generator supports three difficulty levels (EASY, MEDIUM, HARD) which control the noise-to-path connection density, the clique size, and the trap parameters. The highest difficulty (HARD) connects each noise vertex connects to a smaller fraction of path vertices, and more and longer traps are added.

The instance used in the experiments, *generated-hard-5000*, was created with HARD parameters and it contains 5000 vertices, a planted induced path of length 1600, the

noise-to-path density of 22%, 10% inter-clique edges probability and 6 traps of length 500. The resulting graph has 1,310,158 edges and an average degree of approximately 524 which makes it far denser and harder to solve than any of the real-world instances.

3.2 Computational Environment

The algorithms were implemented in Java 21 using the Graph4J for graph storage and manipulation. The experiments ran on a laptop with a 5800H 3.2 GHz CPU and 16 GB of RAM running on Windows 11 Pro 64-bit. Gurobi 13.0.1 was used to run any integer formulation presented in the experiments.

The branch and cut algorithms (CEC and CUT) and the HLIPP heuristics were implemented based on the instructions and pseudocode in [2] and [1] respectively, I've double checked the results I've obtained against those reported in the original publications to verify correctness.

Each algorithm has been given a time limit of 15 minutes per instance. The algorithms that have a random factor involved (GA and ACO) were run 5 times per instance while the deterministic algorithms (CEC, CUT and HLIPP) were run only once. For the genetic algorithm and ACO, I've reported the min, average, median and maximum values for a correct comparison. All algorithms were ran on the same computational environment with the same resources.

3.3 Results

The table below presents the results obtained from running all the algorithms on the 25 test instances. The first four columns represents the details about the instance: the name, number of vertices ($|V|$), the number of edges ($|E|$) and the optimum induced path length (OPT) if it is known. The rest of the columns are the values obtained by the current state of the art algorithms and the maximum values found by the proposed algorithms. If the value is the optimal value then they are shown in **bold**. A dash (–) indicates that the algorithm failed to produce a solution within the time limit.

Table 3.1: Results and running times of all algorithms on all test instances. GA and ACO results are averages over 5 runs, rounded to the nearest integer; times are the average running time. Bold values match the proven optimal.

Instance	$ V $	$ E $	OPT	Marzo et al. [2]				Marzo [1]		Proposed			
				CEC	Time	CUT	Time	HLIPP	Time	GA	Time	ACO	Time
high-tech	33	91	13	13	0.2	13	0.1	13	0.1	13	11.6	13	0.3
karate	34	78	9	9	0.1	9	0.1	9	0.0	9	10.2	9	0.2
mexican	35	117	16	16	0.2	16	0.2	16	0.1	16	13.9	16	0.2
sawmill	36	62	18	18	0.0	18	0.0	18	0.0	18	12.6	18	0.1
taylorS1	39	158	13	13	0.2	13	0.2	13	0.1	13	15.6	13	0.3
chesapeake	39	170	16	16	0.2	16	0.2	16	0.1	16	15.4	16	0.3
taylorS2	39	223	15	15	0.6	15	0.6	15	0.2	15	16.0	15	0.3
romeo and juliet	41	120	9	9	0.0	9	0.0	9	0.0	9	11.0	9	0.2
die hard	47	237	10	10	0.1	10	0.1	10	0.0	10	14.3	10	0.4
attiro	59	128	31	31	0.2	31	0.1	30	1.6	31	26.9	31	0.3
krebs	62	153	17	17	0.2	17	0.2	17	0.2	17	23.2	17	0.3
dolphins	62	159	24	24	0.2	24	0.2	23	1.4	24	28.4	24	0.3
prison	67	142	36	36	0.0	36	0.0	36	2.6	36	32.3	36	0.3
huck	69	297	9	9	0.1	9	0.4	9	0.1	9	20.2	9	0.8
sanjuansur	75	144	38	38	0.2	38	0.1	36	2.6	38	39.6	38	0.3
jean	77	254	11	11	0.1	11	0.2	11	0.3	11	26.2	11	0.6
david	87	406	19	19	0.1	19	0.1	19	2.0	19	36.3	19	1.2
ieeebus	118	179	47	47	0.3	47	0.2	47	8.2	47	62.0	47	0.3
sfi	118	200	13	13	0.0	13	0.0	13	0.1	13	38.1	13	0.8
anna	138	493	20	20	0.2	20	0.2	20	2.5	20	57.8	20	2.2
usair	332	2 126	46	46	123.7	46	104.2	38	50.8	46	242.9	46	16.2
494bus	494	586	142	142	16.6	142	88.5	109	220.9	142	561.7	142	2.5
662bus	662	906	305	305	127.0	305	811.0	237	265.2	277	900.0	301	6.5
yeast	2 361	6 646	?	142	⊖	184	⊖	–	⊖	300	900.0	389	390.1
gen-hard-5000	5 000	1 310 158	$\geq 1600^\dagger$	51	⊖	51	⊖	–	⊖	387	900.0	335	912.7

[†] Lower bound from planted induced path (see Section 3.1.1). Times are in seconds. ⊖ indicates the algorithm timed out at 15 minutes.

3.3.1 Small and medium instances

On all instances up to anna (138 vertices, 493 edges), every algorithm finds the optimal solution. CEC and CUT solve these in under a second in most cases. HLIPP also finds the optimal on most instances but misses it on three: attiro (30 vs 31), dolphins (23 vs 24) and sanjuansur (36 vs 38). The genetic algorithm and the ant colony optimization algorithm find the optimal on all small and medium instances consistently across all 5 runs.

3.3.2 Large real-world instances

The differences between algorithms become apparent on the larger instances. Table 3.2 shows the minimum, median, and maximum results over 5 runs for the stochastic algorithms, along with the average running time.

Table 3.2: Detailed results on large instances. CEC, CUT, and HLIPP are single-run deterministic results. GA and ACO show min/average/max over 5 runs. All times are in seconds.

Instance	OPT	Marzo et al. [2]				Marzo [1]		GA				ACO			
		CEC	Time	CUT	Time	HLIPP	Time	Min	Avg	Max	Time	Min	Avg	Max	Time
usair	46	46	123.7	46	104.2	38	50.8	45	45.8	46	242.9	46	46.0	46	16.2
494bus	142	142	16.6	142	88.5	109	220.9	142	142.0	142	561.7	142	142.0	142	2.5
662bus	305	305	127.0	305	811.0	237	265.2	268	277.0	283	900.0	300	300.6	301	6.5
yeast	?	142	900.5 [†]	184	900.4 [†]	–	900.0 [†]	279	300.2	328	900.0	387	389.2	391	390.1
gen-hard-5000	≥1600	51	918.8 [†]	51	919.2 [†]	–	900.0 [†]	375	386.8	395	900.0	332	334.6	337	912.7

[†] Algorithm timed out at 15 minutes without proving optimality.

On **usair** (332 vertices, 2126 edges), all algorithms find the optimal solution except the HLIPP algorithm. The fastest among these is the ACO which took 16.2 seconds. CUT algorithm took 104 seconds and CEC 123 seconds respectively. The genetic algorithm also managed to obtain the optimum value 4 times out of 5 in 242 seconds.

On **494bus** (494 vertices, 586 edges), CEC gets the optimal value of 142 in 16.6 seconds and CUT in 88.5 seconds. HLIPP finds only 109. Both GA and ACO find the optimal on all 5 runs, with ACO converging in 2.5 seconds on average compared to 561.7 for GA.

On **662bus** (662 vertices, 906 edges), CEC and CUT manage to find the optimal value of 305 which was previously unknown in the literature. CEC finishes in 127 seconds while CUT needs 811 seconds. HLIPP reaches only 237. The best value ACO finds is 301 (with an avg of 300.6) missing the optimal by only 4 vertices. The best value found by the genetic algorithm is 283 further from the optimal. This instance is the last one where CEC and CUT manage to get good results. Till to this point, the CEC and CUT are the best algorithms but adding further vertices and edges start to severely impact the performance of the branch and cut algorithms.

On **yeast** (2361 vertices, 6646 edges), no exact algorithm finishes within the time limit: CEC finds only 142 and CUT finds 184 before timing out. HLIPP fails to produce any valid solution. The best value found by the Ant Colony Optimization algorithm is 391 with an average of 389.2, which is the best known result for this instance. The best found by the GA is 328 with an avg of 300.2, well below ACO which performs the best for this instance.

3.3.3 Generated instance

The generated instance is the hardest instance of them all with 5000 vertices and 1,310,158 edges with an average degree of 524. This is much denser than any of the real world instances and contains many clique structures and a lot of noise. The search space is large but very few of its paths are long.

The induced path planted for this instance is of length 1600. None of the algorithms even came close to this value which shows just how difficult such an instance really is. The longest path found by both the CEC and CUT algorithms is of length 51 before timing out. At this stage, any exact algorithm, even those using integer formulations are no longer feasible. The HLIPP algorithm also failed to provide any results and timed out. The genetic algorithm managed to outperform the ACO algorithm obtaining the maximum value of 395 compared to 337 of the Ant Colony Optimization. This shows that each metaheuristic is best suited for a type of graphs, in this case, the genetic algorithm is better fitted for denser graphs.

3.3.4 Algorithm comparison

Exact algorithms (CEC and CUT). The CEC and CUT algorithms both perform exceptionally well on all instances up to 662bus. The CEC implementation seems to be better as it takes less time. For smaller to medium instances, these two algorithms are the best performing. For larger instances though these algorithms failed to find good results.

HLIPP heuristic. HLIPP finds the optimal for smaller instances but fails to provide good results for medium to large instances. The deterministic path enumeration becomes inefficient on large or dense graphs because the combinatorial explosion prevents it from reaching good regions of the search space.

ACO. The Ant Colony Optimization is the best algorithm found in the experiments for sparse graphs such as the real-world instances used. The direction-based pheromone approach which also takes into account the length of the paths yielded good results. It also efficiently uses all resources by running in parallel as each ant uses one thread of the compute environment. The ACO managed to get the optimum value on all small and medium instances in a short amount of time. For larger instances, ACO managed

to stay competitive on **662bus** obtaining 301, 4 vertices away from the optimal value of 305. The algorithm managed to find the best value found so far for yeast instance obtaining 391 which is much higher than the 328 found by the genetic algorithm or the 184 and 142 values found by the CUT and CEC algorithms respectively.

GA. The genetic algorithm proposed for this problem managed to find the optimal solution for all smaller to medium instances in an average amount of time. For larger instances that are sparse it performed less optimal than ACO as it obtained worse results in the same amount of time. The real value of the genetic algorithm lies in dense graphs, on the hardest instance with 5000 vertices and over 1,310,158 edges, the genetic algorithm managed to outperform all the other algorithms. The longest induced path found by the GA on this instance was 395 compared to the ACO's 337 and the 51 found by CEC and CUT meaning that the genetic algorithm is better on dense graphs.

LLMs were used to help in the making of this thesis. ChatGPT 5.5 was used to generate the various images used as explanations for the algorithms. Claude 4.6 Opus has been used to generate code, test code and to provide ideas. All ideas and many modifications have been made by me and most of this thesis is written by me with Claude only used to slightly rephrase parts of it.

Chapter 4

Concluding remarks

4.1 Ethical AI Usage

LLMs were used to help in the making of this thesis. ChatGPT 5.5 was used to generate the various images used as explanations for the algorithms. Claude 4.6 Opus has been used to generate code, test code, provide ideas and scrape the internet for any useful information that can be used to solve this problem. All ideas and many modifications have been made by me and most of this thesis is written by me with Claude 4.6 Opus used only to rephrase parts of it.

4.2 Conclusion

In this thesis, I proposed two metaheuristic algorithms for the longest induced path problem: an ant colony optimization algorithm and a genetic algorithm. To the best of my knowledge, these are the first metaheuristics proposed for this problem. The existing literature relies on exact algorithms, integer linear programming formulations and simple heuristics based on path enumeration, none of which scale well to large or dense graphs.

Both algorithms had to be designed specifically for this problem. The main challenge was maintaining the induced path property throughout the search as standard metaheuristic operators have a high chance of producing invalid solutions. I addressed this through the *invalidCount* array mechanism, which tracks how many neighbors each vertex has within the current path and allows $O(1)$ feasibility checks when extending the path. This mechanism is shared by both algorithms and is what makes the

mutation, crossover and path construction operators work correctly without expensive repair steps.

The ACO algorithm uses a pheromone-guided depth-first search with backtracking instead of the standard forward-only walk used in classical ACO applications. The pheromone model uses directed edge pheromones, meaning the pheromone value from vertex v to u is independent from the value from u to v . This allows the algorithm to use the direction of traversal which matters a lot when building induced paths. The addition of adaptive evaporation, pheromone restarts and tabu lists also improved the performance of the algorithm significantly.

The genetic algorithm uses an ordered vertex array representation compared to my first iteration from my bachelor thesis that used a boolean array. This change improved the efficiency of all operators significantly. The mutation operator trims vertices from both ends of the path and re-extends while the crossover combines the vertex sets of two parents and finds an induced path within the union. Both operators use a modified DFS to find locally maximal paths.

The computational experiments showed that each algorithm class has its strengths. The exact branch-and-cut algorithms CEC and CUT from Marzo et al. [2] are the best choice for small to medium instances as they provide provably optimal solutions in reasonable time. However, on large instances, these algorithms fail to execute within the time limit and the partial results that they find are not competitive compared to the genetic algorithm and ACO.

The HLIPP heuristic from Marzo and Ribeiro [1] finds the optimal on most small instances but consistently falls short on larger ones. It takes more time than the other algorithms and it fails to provide good results on any of the larger instances.

The ACO algorithm proved to be the strongest overall performer on sparse real-world instances. It found the optimal value on all instances up to 494bus, obtained 301 on 662bus (4 vertices from the proven optimal of 305), and achieved the best known result of 391 on the yeast instance. ACO also showed the fastest convergence, finding the optimal on 494bus in 2.5 seconds while the exact algorithms needed over 16 seconds. The pheromone-guided search works well on sparse graphs where the pheromone trails can encode meaningful directional information about promising paths.

The genetic algorithm showed its value on the dense generated instance with 5000 vertices and over 1.3 million edges. On this instance, the GA found paths of up to 395 vertices compared to ACO's 337, CEC and CUT's 51 and HLIPP's failure

to produce any result. The population-based approach with crossover handles dense graphs better than ACO's pheromone-guided search, as the high vertex degree dilutes the pheromone signal in the ACO. Both proposed algorithms significantly improved upon the results from my bachelor thesis, with the most notable improvement being on the yeast instance (from 245 to 391 with ACO).

The instance generator I developed for this thesis fills a gap in the experimental methodology for this problem. By producing graphs with a planted induced path of known length, it provides a lower bound on the optimal solution that allows measuring how far each algorithm is from the true optimum. The generated instance demonstrated that even the best algorithms recover less than 25% of the planted path on dense graphs, highlighting the difficulty of the problem and the room for improvement.

4.3 Future Work

There are several directions that could improve upon the results obtained in this thesis:

- **Hybrid algorithms.** Combining the strengths of both metaheuristics could yield better results. For example, using ACO to generate an initial population for the GA, or using GA's crossover to combine paths found by different ACO runs. The ILP formulations could also be used to seed the metaheuristics with good initial solutions on instances where they can find reasonable results within a short time limit. We could also attempt to run a few iterations of ACO on the population of the genetic algorithm after each generation.
- **Adaptive algorithm selection.** Since ACO performs better on sparse graphs and GA on dense ones, an algorithm that detects the graph structure and selects the appropriate metaheuristic (or blends them) could provide the best results across all instance types.
- **Local search operators.** Adding a memetic component to the GA, where each individual undergoes a local search step after mutation or crossover, could improve solution quality. Similarly, the ACO could benefit from a local search phase applied to the best paths found in each iteration.

- **Larger and more diverse test instances.** The instance generator could be extended to produce sparse graphs similar to real-world networks with known planted paths. Testing on a wider range of graph families (random graphs, scale-free networks, planar graphs) would give a more complete picture of each algorithm's strengths and weaknesses.
- **Parallelism improvements.** The current ACO implementation runs each DFS task in parallel but performs pheromone updates sequentially. A lock-free or partitioned pheromone update scheme could improve scalability on machines with many cores.

Bibliography

- [1] Marzo, Ruslán G., and Celso C. Ribeiro. "Exact and approximate algorithms for the longest induced path problem." *RAIRO-Operations Research* 55.2 (2021): 333-353.
- [2] Marzo, R. G., Melo, R. A., Ribeiro, C. C., & Santos, M. C. (2022). New formulations and branch-and-cut procedures for the longest induced path problem. *Computers & Operations Research*, 139, 105627.
- [3] Magnanti, T. L., & Wolsey, L. A. (1995). Optimal trees. *Handbooks in operations research and management science*, 7, 503-615.
- [4] D. Matsypura, A. Veremyev, O.A. Prokopyev, and E.L. Pasilio, "On exact solution approaches for the longest induced path problem," *Eur. J. Oper. Res.*, vol. **278**, no. 3, pp. 546–562, 2019.
- stutzle2000 Stützle, T., Hoos, H. H. (2000). MAX-MIN ant system. *Future generation computer systems*, 16(8), 889-914.
- [5] Johnson, David S., and Michael R. Garey. *Computers and intractability: A guide to the theory of NP-completeness*. WH Freeman, 1979.
- [6] Kautz, William H. "Unit-distance error-checking codes." *IRE Transactions on Electronic Computers* 2 (1958): 179-180.
- [7] Bökler, F., Chimani, M., Wagner, M.H., and Wiedera, T., "An experimental study of ILP formulations for the longest induced path problem." In *Combinatorial Optimization*, edited by M. Baiou, B. Gendron, O. Günlük, and A.R. Mahjoub. *Lecture Notes in Computer Science*, Springer, Cham (2020), Vol. 12176.
- [8] Gavril, F. "Algorithms for maximum weighted induced paths." *Inf. Process. Lett.* 81 (2002): 203-208.

- [9] Isheziki, T., Otachi, Y., and Yamazaki, K. "An improved algorithm for the longest induced path problem on k-chordal graphs." *Discrete Appl. Math.* 156 (2008): 3057-3059.
- [10] Jaffke, L., Kwon, O.-J., and Telle, J. "Polynomial-time algorithms for the Longest Induced Path and Induced Disjoint Paths problems on graphs of bounded mim-width." In *Leibniz International Proceedings in Informatics* (2018): 1-21.
- [11] Jaffke, L., Kwon, O.-J., and Telle, J.A. "Min-Width I. Induced path problems." *Discrete Appl. Math.* 278 (2020): 153-168.
- [12] Kratsch, D., Müller, H., and Todinca, I. "Feedback vertex set and longest induced path on AT-free graphs." In: *International Workshop on Graph-Theoretic Concepts in Computer Science*. Springer (2003): 309-321.
- [13] Bergougnoux, B., and Kanté, M.M. "A meta-algorithm for solving connectivity and acyclicity constraints on locally checkable properties parameterized by graph width measures." Preprint arXiv:1805.11275 (2018).
- [14] PassMark Software. "CPU Benchmark Comparison: Intel Core i5-4460S vs. AMD Ryzen 7 5800H." Available from: <https://www.cpubenchmark.net/compare/2232vs3907/Intel-i5-4460S-vs-AMD-Ryzen-7-5800H> (2026).